



ĐỒ ÁN TỐT NGHIỆP KỸ SƯ

Đồ án tốt nghiệp kỹ sư (Trường Đại học Bách khoa Hà Nội)



Scan to open on Studocu

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Phát triển hệ thống Video Conferencing

NGUYỄN BÁ HOÀNG

hoang.nb183543@sis.hust.edu.vn

Ngành Khoa học máy tính

Giảng viên hướng dẫn: TS. Phạm Quang Dũng

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 08/2022

LỜI CẢM ƠN

Đầu tiên, em xin gửi lời cảm ơn chân thành đến các thầy cô trong trường Đại học Bách Khoa Hà Nội, các thầy cô trong trường Công nghệ thông tin và Truyền thông, trong phòng Đào tạo đã hướng dẫn, chỉ bảo em trong 4 năm học tập tại trường.

Đặc biệt hơn, với tình cảm sâu sắc và chân thành nhất, cho phép em được gửi lời cảm ơn đến thầy ***Phạm Quang Dũng*** vì đã hướng dẫn, hỗ trợ em trong suốt quá trình học tập, nghiên cứu, phát triển đề tài này. Nhờ sự chỉ bảo tận tâm từ thầy mà em có cơ hội tiếp xúc, làm chủ với các công nghệ mới, làm việc và trải nghiệm trong một dự án thực tế.

Một lần nữa, em xin chân thành cảm ơn!

TÓM TẮT NỘI DUNG ĐỒ ÁN

Ngày nay, dù đại dịch Covid đã đi qua, nhưng các hoạt động xã hội như học tập, làm việc, trò chuyện từ xa vẫn phát triển mạnh mẽ. Video Call trở thành một phần quan trọng trong cuộc sống, các buổi học, các cuộc họp, những buổi trò chuyện có thể diễn ra trực tuyến. Video Call là một công nghệ đã có từ lâu, nhưng vẫn chưa hoàn thiện và vẫn đang được đổi mới từng ngày. Hiện nay, nhiều hệ thống lớn đã được phát triển và sử dụng rộng rãi như Microsoft Teams [1], Zoom[2], Google Meet [3] hay là một phần của mạng xã hội như Zalo [4], Facebook [5],... Việc làm chủ công nghệ và từ đó thiết kế và phát triển các tính năng mới, đặc thù cũng là nhu cầu của nhiều tổ chức, doanh nghiệp.

Trong đồ án này, chúng tôi thiết kế và phát triển một hệ thống Video Conference. Hệ thống này là một phần của dự án open source về ERP [6]. Hệ thống được phát triển dựa trên công nghệ WebRTC, kết nối P2P để đảm bảo tính thời gian thực cho cuộc gọi, tận dụng tốt phần cứng của người dùng. Hiện tại, hệ thống đã hỗ trợ những tính năng cơ bản như nhắn tin, gọi điện, chia sẻ màn hình, quản lý danh sách cuộc gọi,...

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	1
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	2
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	4
2.1 Khảo sát hiện trạng	4
2.2 Tổng quan chức năng	4
2.2.1 Biểu đồ use case tổng quát	5
2.2.2 Biểu đồ use case phân rã quản lý danh sách VC của bản thân	6
2.2.3 Biểu đồ use case phân rã quản lý danh sách người tham gia VC.....	7
2.2.4 Quy trình nghiệp vụ	7
2.3 Đặc tả chức năng	10
2.3.1 Đặc tả chức năng lên lịch một VC	10
2.3.2 Đặc tả chức năng xem danh sách VC của bản thân.....	11
2.3.3 Đặc tả chức năng chia sẻ camera, micro	12
2.3.4 Đặc tả chức năng chia sẻ màn hình	13
2.3.5 Đặc tả chức năng mời người dùng khác tham gia vào VC.....	14
2.4 Yêu cầu phi chức năng	14
CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG.....	15
3.1 Công nghệ kế thừa	15
3.1.1 Java	15
3.1.2 Springboot	15
3.1.3 ReactJs.....	16

3.1.4 PostgreSQL	16
3.2 Công nghệ đóng góp	16
3.2.1 WebSocket	16
3.2.2 WebRTC	17
3.2.3 NodeJs	17
3.2.4 PeerJs và PeerServer.....	17
CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ	18
4.1 Thiết kế kiến trúc.....	18
4.1.1 Lựa chọn kiến trúc phần mềm	18
4.1.2 Thiết kế tổng quan.....	19
4.1.3 Thiết kế chi tiết gói	21
4.2 Thiết kế chi tiết.....	23
4.2.1 Thiết kế giao diện	23
4.2.2 Thiết kế lớp	24
4.2.3 Thiết kế cơ sở dữ liệu	27
4.3 Xây dựng ứng dụng.....	30
4.3.1 Thư viện và công cụ sử dụng.....	30
4.3.2 Kết quả đạt được	31
4.3.3 Minh họa các chức năng chính	31
4.4 Kiểm thử.....	35
4.5 Triển khai	36
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT.....	38
5.1 Chia sẻ camera, micro, màn hình thời gian thực với người khác	38
5.2 WebRTC.....	40
5.2.1 MediaStream	40
5.2.2 RTCPeerConnection.....	41

5.2.3 RTCDataChannel.....	42
5.3 P2P và ICE	43
5.4 Mô hình Video Call thông qua WebRTC.....	44
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	47
6.1 Kết luận.....	47
6.2 Hướng phát triển.....	47
TÀI LIỆU THAM KHẢO.....	49

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát	5
Hình 2.2	Biểu đồ use case phân rã quản lý danh sách VC của bản thân	6
Hình 2.3	Biểu đồ use case phân rã quản lý danh sách người tham gia VC	7
Hình 2.4	Biểu đồ hoạt động minh họa quy trình tạo, sửa, xóa cuộc gặp, mời người dùng khác tham gia	8
Hình 2.5	Biểu đồ hoạt động minh họa quy trình tham gia cuộc gặp, nhắn tin, chia sẻ file, micro, camera, màn hình và rời cuộc gặp của người dùng	9
Hình 4.1	Mô hình ba tầng	18
Hình 4.2	Biểu đồ phụ thuộc gói	19
Hình 4.3	Biểu đồ thiết kế gói Model	21
Hình 4.4	Biểu đồ thiết kế gói Controller, Service, Repository	22
Hình 4.5	Thiết kế màn hình trang chủ	23
Hình 4.6	Thiết kế màn hình cuộc gặp	24
Hình 4.7	Thiết kế lớp RoomParticipantController	24
Hình 4.8	Thiết kế lớp RoomParticipantService	25
Hình 4.9	Thiết kế lớp RoomParticipantRepository	25
Hình 4.10	Biểu đồ trình tự tham gia một cuộc gặp	26
Hình 4.11	Biểu đồ trình tự mời người dùng khác tham gia vào cuộc gặp	27
Hình 4.12	Biểu đồ thực thể liên kết (E-R Diagram)	27
Hình 4.13	Biểu đồ thiết kế cơ sở dữ liệu	28
Hình 4.14	Giao diện trang chủ	31
Hình 4.15	Giao diện lên lịch một cuộc gặp	32
Hình 4.16	Giao diện danh sách cuộc gặp của bạn	32
Hình 4.17	Giao diện Tham gia cuộc gặp khác	33
Hình 4.18	Giao diện màn hình cuộc gặp	33
Hình 4.19	Giao diện màn hình cuộc gặp với thẻ quản lý danh sách người tham gia	34
Hình 5.1	Media Stream	40
Hình 5.2	WebRTC architecture	41
Hình 5.3	Interactive Connectivity Establishment (ICE)	43
Hình 5.4	Đăng ký và chia sẻ mã định danh peerID	44
Hình 5.5	Kết nối P2P và chia sẻ luồng dữ liệu đa phương tiện với những người tham gia khác trong cuộc gặp	45

DANH MỤC BẢNG BIỂU

Bảng 2.1	Đặc tả chức năng lên lịch một VC	10
Bảng 2.2	Các trường trong phần tạo cuộc gặp	10
Bảng 2.3	Đặc tả chức năng xem danh sách VC của bản thân	11
Bảng 2.4	Đặc tả chức năng chia sẻ camera, micro	12
Bảng 2.5	Đặc tả chức năng chia sẻ màn hình	13
Bảng 2.6	Đặc tả chức năng mời người dùng khác tham gia vào VC	14
Bảng 4.1	Mục đích/Nhiệm vụ của các gói trong hệ thống	20
Bảng 4.2	Danh sách và ý nghĩa các bảng trong cơ sở dữ liệu	29
Bảng 4.3	Bảng Room	29
Bảng 4.4	Bảng User	29
Bảng 4.5	Bảng RoomParticipant	29
Bảng 4.6	Danh sách thư viện và công cụ sử dụng	30
Bảng 4.7	Thống kê một số thông tin của sản phẩm	31
Bảng 4.8	Các trường hợp kiểm thử cho chức năng tạo cuộc gặp	35
Bảng 4.9	Các trường hợp kiểm thử cho chức năng mời người dùng tham gia cuộc gặp	36
Bảng 4.10	Tổng hợp các trường hợp kiểm thử	36
Bảng 4.11	Thông số và cấu hình server	37

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
Browser	Trình duyệt
CSS	Cascading Style Sheets
Deploy	Triển khai
ERP	Hoạch định nguồn lực doanh nghiệp (Enterprise Resource Planning)
Framework	Nền tảng lập trình ứng dụng
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
HTTP	Giao thức truyền tải siêu văn bản (Hypertext Transfer Protocol)
HTTPS	Giao thức truyền tải siêu văn bản an toàn (Hypertext Transfer Protocol Secure)
Open Source	Mã nguồn mở
P2P	Ngang hàng (peer-to-peer)
VC	Cuộc gặp trực tuyến (Video Conference)
WebRTC	Giao tiếp thời gian thực trên web (Web Realtime Communication)
WSS	Giao thức kết nối Web Socket an toàn (WebSocket Secure)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Học tập, trò chuyện và làm việc từ xa chắc hẳn không còn quá xa lạ trong xã hội hiện tại. Nếu trước kia, đây chỉ là sự lựa chọn dành cho những ai bận rộn, không có khả năng gặp trực tiếp, thì hiện nay, điều này đã trở thành xu hướng của xã hội. Đại dịch Covid 19 đã thay đổi hoàn toàn thói quen học tập, trò chuyện, làm việc của chúng ta. Với sự phát triển mạnh mẽ của công nghệ thông tin, học tập, trò chuyện và làm việc trực tuyến là một giải pháp giúp kết nối chúng ta lại với nhau. Nhiều nền tảng lớn đã phục vụ cho việc này như Microsoft Teams [1], Zoom [2], Google Meet [3],... với những chính sách riêng về người dùng, chi phí sử dụng dịch vụ,... Tại đây người dùng sẽ đăng nhập, tạo Video Conference (VC), cùng nhau tham gia vào VC, bật microphone, camera hoặc chia sẻ màn hình, nhắn tin để có thể trao đổi trực tuyến với nhau. Việc kết nối, tương tác và tích hợp vào các hệ thống khác không phải lúc nào cũng khả thi và thuận lợi. Với những yêu cầu đặc thù của giáo viên và sinh viên của trường, cũng như những nhu cầu phát sinh trong quá trình học tập và làm việc không phải lúc nào cũng được đáp ứng. Việc làm chủ công nghệ và từ đó thiết kế giải pháp với các ý tưởng riêng, đáp ứng các yêu cầu đặc thù luôn được các tổ chức doanh nghiệp (đặc biệt là doanh nghiệp lớn) quan tâm.

Trong đồ án này, chúng tôi thiết kế và xây dựng hệ thống Video Conference (VC) với các tính năng cho phép người dùng lên lịch, tạo, quản lý các VC, mời người khác tham gia vào VC. Người dùng trong cùng một cuộc họp có thể nhắn tin, bật micro, camera hoặc chia sẻ màn hình với nhau, xem danh sách người dùng đang có trong cuộc họp, thông báo khi có người dùng tham gia, rời khỏi VC. Hệ thống này được phát triển là một phần trong dự án mã nguồn mở về ERP [6].

1.2 Mục tiêu và phạm vi đề tài

Hiện nay, hệ thống VC đã hoàn thành toàn bộ các chức năng đã nói ở trên, đảm bảo trải nghiệm tốt (đạt toàn bộ các yêu cầu phi chức năng được nêu ở phần 2.4) cho người dùng. Nhìn chung sản phẩm đã đáp ứng được nhu cầu cơ bản của người sử dụng, có thể thay thế các sản phẩm Video Conference hiện có trong các cuộc họp với số lượng người tham gia nhỏ. Sản phẩm cần bổ sung thêm một số chức năng như record lại cuộc gặp, phân quyền người dùng trong một VC (chủ phòng có quyền cho phép hoặc không cho phép người dùng khác tham gia vào cuộc gặp của mình, có quyền mời một người dùng khác ra khỏi phòng,...), tăng số lượng tối đa người tham gia cho một cuộc gặp.

1.3 Định hướng giải pháp

Đối với các yêu cầu chức năng lên lịch, tạo, quản lý các VC, mời người khác tham gia vào VC, xem danh sách người dùng đang có trong cuộc họp chúng tôi sử dụng chuẩn giao tiếp RESTful API [7]. Các chức năng nhắn tin, bật micro, camera hoặc chia sẻ màn hình với nhau, thông báo khi có người dùng tham gia, rời khỏi VC yêu cầu độ trễ thấp, thời gian thực chúng tôi dùng WebSocket, kết hợp cùng công nghệ WebRTC, kết nối P2P thông qua thư viện PeerJS để truyền nhận tín hiệu, luồng dữ liệu âm thanh, hình ảnh của người dùng. Với yêu cầu chức năng record lại cuộc gặp, chúng tôi sẽ trình bày hình ảnh, âm thanh từ camera, micro, màn hình của những người dùng trong cuộc gặp trên thẻ canvas của HTML. Khi người dùng chọn chức năng record thì hệ thống sẽ sử dụng phương thức captureStream trên thẻ canvas của Javascript để quay lại cuộc gặp, tạo thành luồng video rồi lưu lại. Đối với chức năng phân quyền người dùng trong một cuộc gọi, chúng tôi sẽ sử dụng WebSocket, tạo và xử lý các sự kiện người dùng khác yêu cầu tham gia vào cuộc gọi, mời người dùng khác ra khỏi phòng. Yêu cầu tăng số lượng tối đa cho một cuộc họp là yêu cầu tương đối khó khăn, kích thước màn hình của mỗi người dùng là cố định và khác nhau, tương tự với kích thước của hình ảnh lấy từ camera của từng người dùng cũng là khác nhau. Bài toán đặt ra là khi số lượng người dùng trong phòng tăng lên và cùng bật camera, chia sẻ màn hình thì hệ thống cần hiển thị như thế nào cho đẹp mắt, cân đối (Ví dụ khi có 6 người thì hiển thị thành 2 hàng mỗi hàng 3 luồng hình ảnh).

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày về những khảo sát nhu cầu người dùng và hiện trạng các hệ thống tương tự đối với vấn đề được nêu tại phần 1.1. Sau khi phân tích, chúng tôi tìm ra những vấn đề cần được giải quyết, quyết định các tính năng sẽ được phát triển cho hệ thống. Qua đó, chúng tôi đưa ra những mô tả tổng quan nhất về hệ thống, về yêu cầu chức năng, yêu cầu phi chức năng và về quy trình nghiệp vụ.

Từ những phân tích của chương 2, chương 3 trình bày về những công nghệ được sử dụng trong hệ thống. Chương 3 được chia làm 2 phần, phần một trình bày cập tới các công nghệ kế thừa từ dự án mã nguồn mở ERP, phần hai trình bày về những công nghệ được tìm hiểu và áp dụng vào hệ thống. Ứng với mỗi công nghệ được sử dụng vào hệ thống, chúng tôi có những phân tích rõ ràng công dụng của công nghệ này, và trả lời một số câu hỏi như tại sao lại sử dụng nó, sử dụng nó để giải quyết vấn đề gì, ngoài công nghệ này còn có thể dùng công nghệ nào khác thay thế, ưu nhược điểm khi sử dụng những công nghệ thay thế đó.

Dựa vào những yêu cầu được nêu ở chương 2 và những công nghệ được áp dụng ở chương 3, phần đầu chương 4 trình bày về thiết kế kiến trúc, thiết kế chi tiết của hệ thống. Phần tiếp theo đề cập tới việc xây dựng ứng dụng bao gồm các thư viện và công cụ sử dụng; những kết quả đạt được; minh họa những chức năng chính. Phần cuối của chương là những thông tin, thống kê, kết quả kiểm thử, việc triển khai và trải nghiệm thực tế của người dùng. Nội dung chương 4 thể hiện rõ hệ thống làm gì, đã làm được gì, chất lượng sản phẩm.

Tiếp theo, chương 5 trình bày về những nội dung đóng góp mà chúng tôi thấy tâm đắc nhất, những công nghệ cốt lõi cho bài toán truyền nhận dữ liệu âm thanh, hình ảnh thời gian thực giữa một nhóm nhiều người, những khó khăn gặp phải, những giải pháp giải quyết khó khăn trong suốt quá trình làm DATN. Chương này thể hiện rõ những kiến thức đã áp dụng, cũng như học được khi làm DATN.

Cuối cùng, chương 6 là chương tổng kết lại toàn bộ DATN này, so sánh kết quả sản phẩm của chúng tôi với các sản phẩm hiện có. Đồng thời, chương này cũng trình bày về định hướng cho sản phẩm trong tương lai để hoàn thiện hơn.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

2.1 Khảo sát hiện trạng

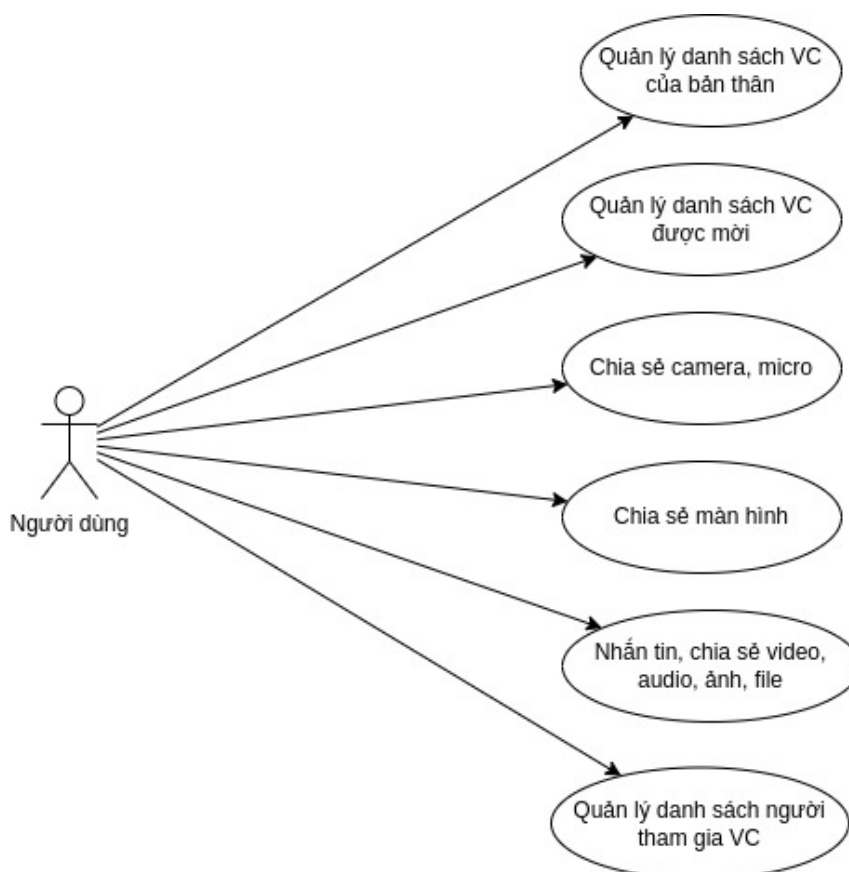
Các sản phẩm về Video Call có mặt phổ biến trên thị trường như Google Meet [3], Microsoft Teams [1], Facebook [5], Zoom [2],... Đây đều là sản phẩm của những công ty, tập đoàn công nghệ hàng đầu trên thế giới, mỗi sản phẩm đều có những ưu, nhược điểm riêng. Ví dụ như với Microsoft Teams, đây là một công cụ làm việc nhóm vô cùng mạnh mẽ ngoài Video Call còn hỗ trợ rất nhiều tính năng khác, tuy nhiên người dùng cần có tài khoản Microsoft và cần nhiều thao tác phức tạp để có thể truy cập, tạo và tham gia vào một cuộc gặp trực tuyến. Hay với Google Meet, đây là một công cụ về Video Call vô cùng mạnh mẽ, cách sử dụng đơn giản, hoàn toàn miễn phí, hỗ trợ lên tới 100 người trong một cuộc gặp [8]. Đối với người dùng của dự án ERP [6] khi muốn gặp mặt trực tuyến với nhau thì một người cần vào các hệ thống trên để tạo một cuộc gặp, sau đó thông qua các kênh nhắn tin khác, tìm người muốn mời tham gia và chia sẻ link cuộc gặp. Điều này là khá tốn kém thời gian và công sức của người dùng hệ thống ERP, cũng như các yêu cầu đặc thù của người dùng trong tương lai không phải lúc nào cũng được đáp ứng nếu sử dụng những sản phẩm trên. Chính vì vậy, chúng tôi quyết định phát triển hệ thống Video Conference cho dự án ERP để phục vụ một cách đơn giản, nhanh cho người dùng của dự án. Sản phẩm lấy cảm hứng từ Google Meet, giữ lại những tính chất quan trọng như hoàn toàn miễn phí, thao tác đơn giản, độ trễ khi truyền nhận luồng video, âm thanh thấp.

2.2 Tổng quan chức năng

Sau khi thu thập ý kiến của bạn bè (những người đã học tập trực tuyến hàng ngày), cũng như dựa trên những trải nghiệm đã có khi học tập online của bản thân chúng tôi trên nền tảng Microsoft Teams, Zoom, một hệ thống gặp trực tuyến cần có những chức năng cơ bản như sau:

- (i) Hỗ trợ tính năng lên lịch và tạo VC.
- (ii) Quản lý danh sách VC của bản thân.
- (iii) Tham gia vào một VC.
- (iv) Chia sẻ micro, camera, màn hình của bản thân với những người tham gia cùng một VC.
- (v) Nhắn tin, chia sẻ ảnh, video, audio, file cho những người trong cuộc họp.
- (vi) Quản lý danh sách những người tham gia vào VC.

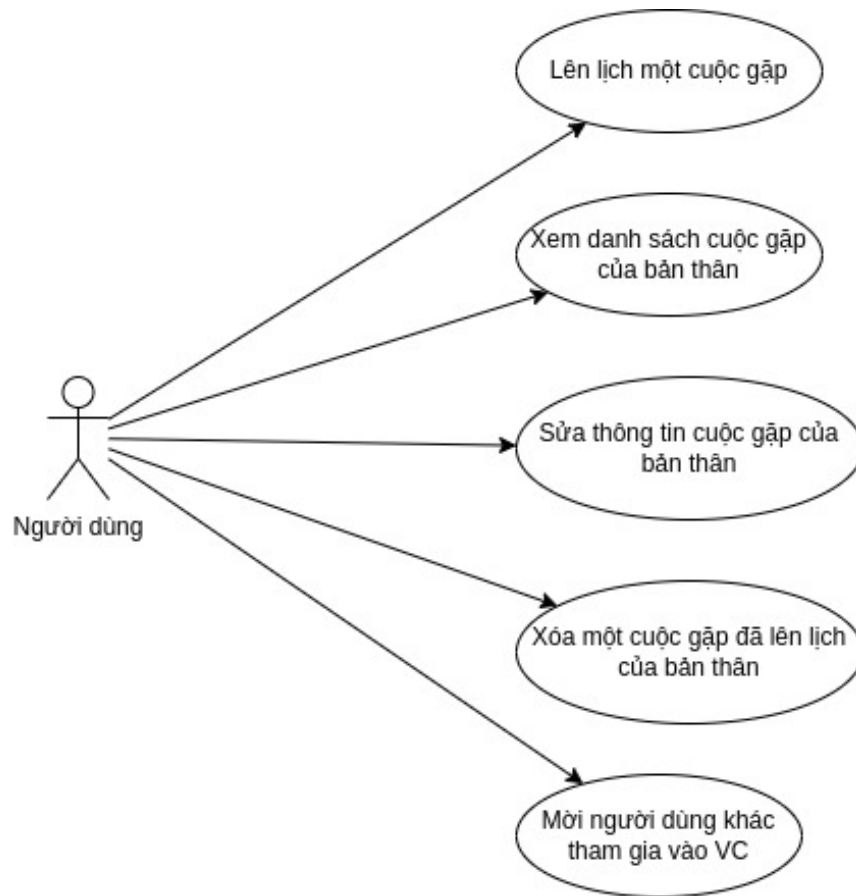
2.2.1 Biểu đồ use case tổng quát



Hình 2.1: Biểu đồ use case tổng quát

Hình 2.1 là biểu đồ use case tổng quát của hệ thống với các use case chính là quản lý danh sách VC của bản thân, quản lý danh sách VC được mời, chia sẻ camera, micro, màn hình của bản thân trong một cuộc gặp mặt trực tuyến, tác nhân duy nhất là người dùng - bất cứ người dùng nào được phân quyền sử dụng module Video Conference đều được coi là tác nhân người dùng. Với use case quản lý danh sách VC của bản thân, một người dùng có thể xem danh sách VC của bản thân, tạo mới, sửa tên, thời gian bắt đầu, kết thúc của một cuộc gặp, tham gia vào VC đã lên lịch và xóa. Với use case quản lý danh sách VC được mời, người dùng có thể xem danh sách VC mà mình được mời, tham gia vào VC trong danh sách đó. Use case chia sẻ camera, micro, màn hình mô tả cách người dùng chia sẻ camera, micro, màn hình của bản thân với những người dùng khác trong cùng một cuộc họp.

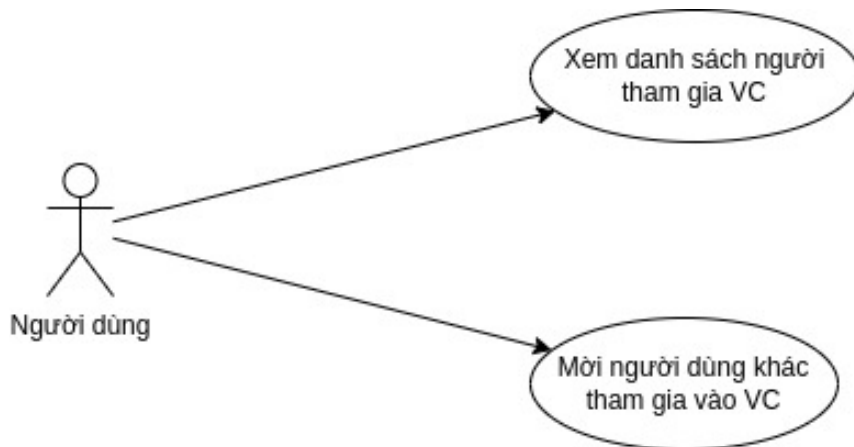
2.2.2 Biểu đồ use case phân rã quản lý danh sách VC của bản thân



Hình 2.2: Biểu đồ use case phân rã quản lý danh sách VC của bản thân

Hình 2.3 là biểu đồ use case phân rã quản lý danh sách VC của bản thân. Người dùng có thể lên lịch một cuộc gặp, xem danh sách VC của bản thân, sửa thông tin các cuộc gặp trong danh sách đó (sửa tên, sửa thời gian bắt đầu, thời gian kết thúc cuộc họp), xóa những cuộc gặp do không cần thiết hoặc lên lịch nhầm và mời người khác tham gia vào cuộc gặp.

2.2.3 Biểu đồ use case phân rã quản lý danh sách người tham gia VC

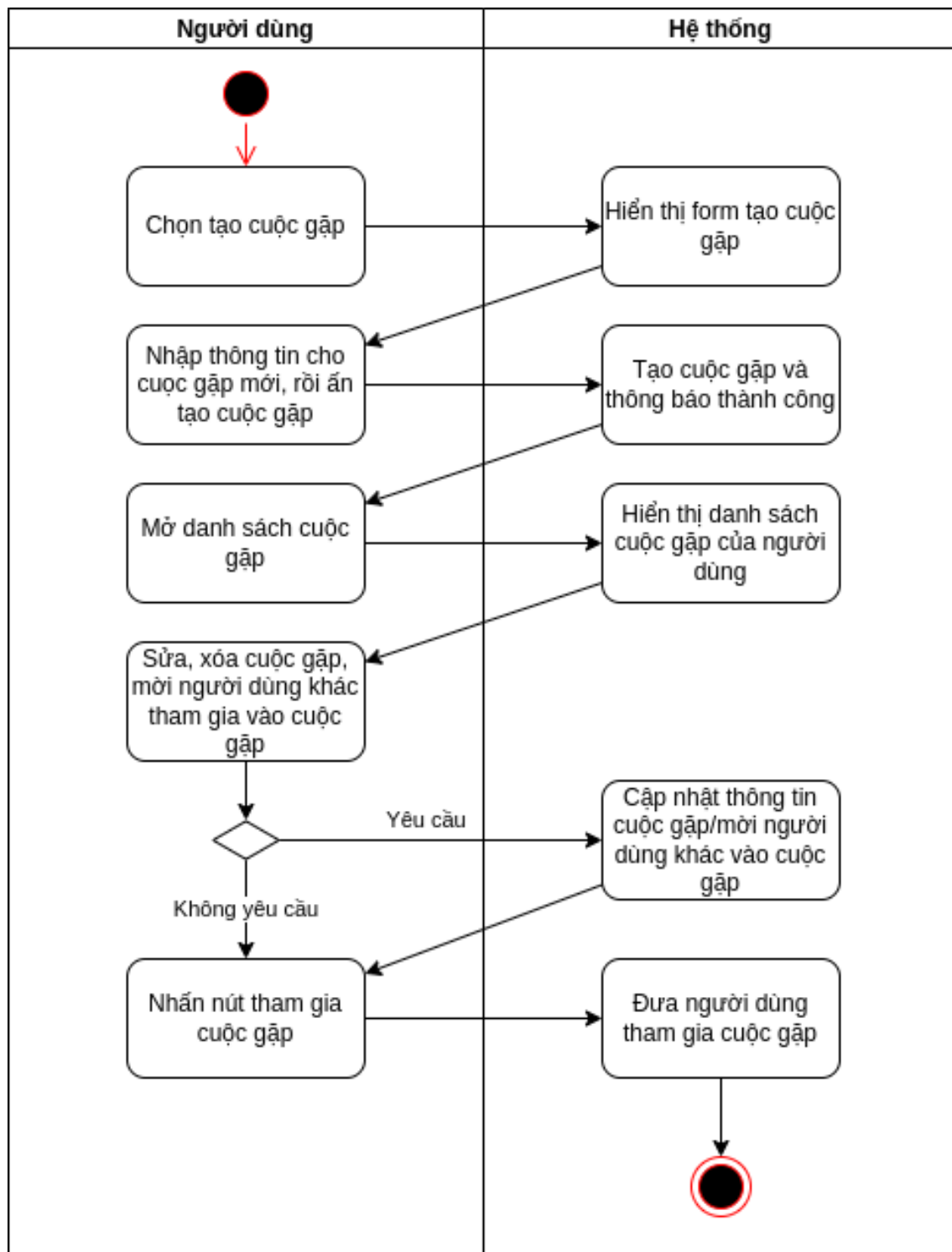


Hình 2.3: Biểu đồ use case phân rã quản lý danh sách người tham gia VC

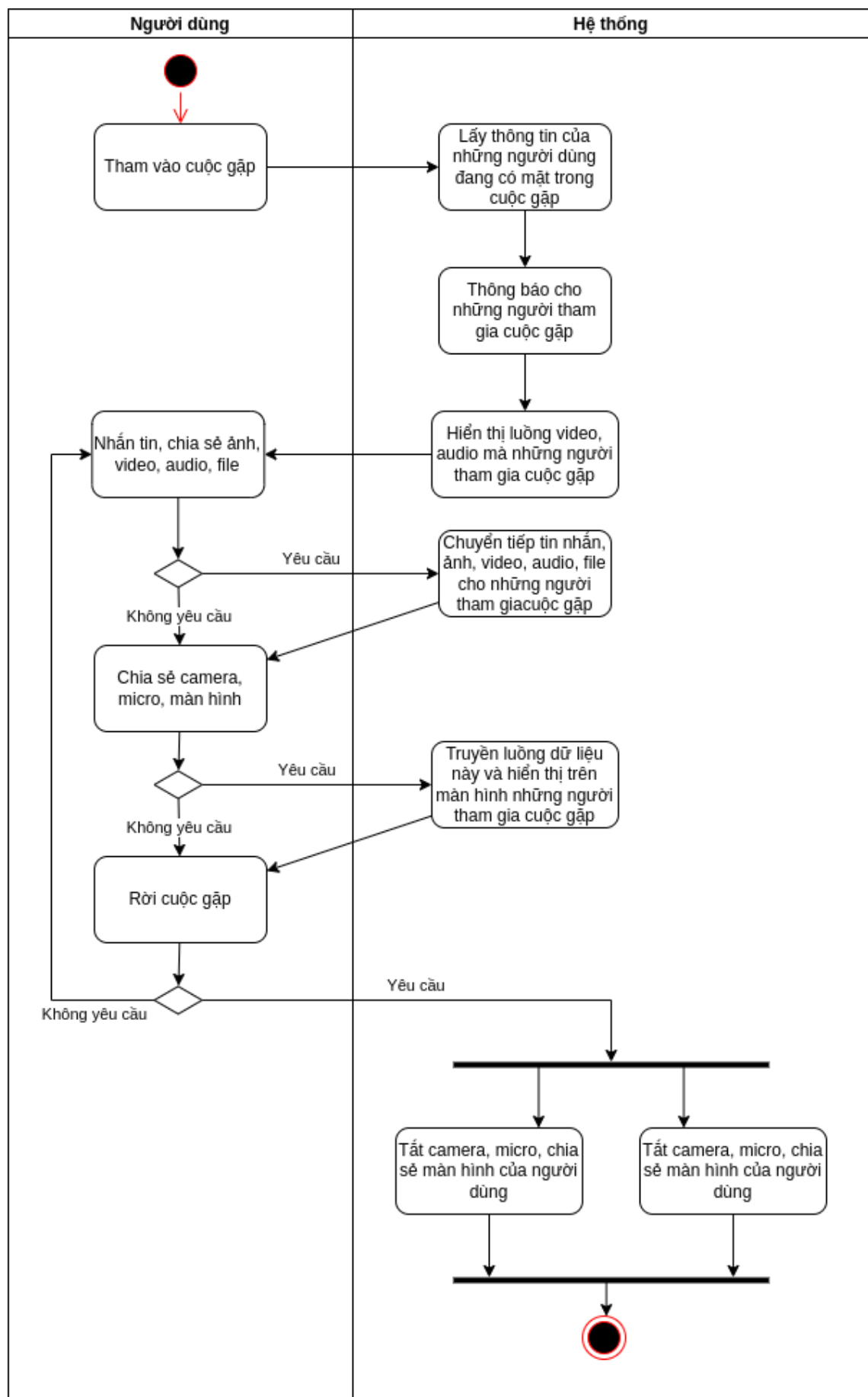
Hình 2.3 là biểu đồ use case phân rã quản lý danh sách người tham gia VC. Người dùng bất kỳ trong cuộc gặp đều có thể xem danh sách những người đang có mặt trong cuộc gặp và mời một người dùng bất kỳ trong hệ thống tham gia vào cuộc gặp này.

2.2.4 Quy trình nghiệp vụ

Dưới đây là 2 biểu đồ hoạt động minh họa cho các chức năng chính của hệ thống bao gồm tạo, sửa, xóa cuộc gặp, mời người dùng khác tham gia; tham gia vào cuộc gặp, nhắn tin, chia sẻ file, micro, camera, màn hình và rời cuộc gặp của người dùng.



Hình 2.4: Biểu đồ hoạt động minh họa quy trình tạo, sửa, xóa cuộc gặp, mời người dùng khác tham gia



Hình 2.5: Biểu đồ hoạt động minh họa quy trình tham gia cuộc gặp, nhắn tin, chia sẻ file, micro, camera, màn hình và rời cuộc gặp của người dùng

2.3 Đặc tả chức năng

2.3.1 Đặc tả chức năng lên lịch một VC

Mã ca sử dụng	UC01
Tên ca sử dụng	Lên lịch một VC
Tên tác nhân	Người dùng
Mô tả	Ca sử dụng mô tả cách người dùng lên lịch một VC
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống và được phân quyền sử dụng module VC
Luồng sự kiện chính	1. Chọn thẻ "Quản lý cuộc gặp của bản thân" 2. Nhấn "Lên lịch một cuộc họp" 3. Nhập tên của VC 4. Chọn thời gian bắt đầu và kết thúc của VC. 5. Nhấn lên lịch "Lên lịch" 6. Kiểm tra kết quả trả về từ hệ thống là lên lịch thành công hay thất bại
Hậu điều kiện	Không có
Luồng sự kiện thay thế	Không có

Bảng 2.1: Đặc tả chức năng lên lịch một VC

STT	Trường dữ liệu	Yêu cầu	Mô tả	Định dạng
1	Tên	Có	Tên cuộc gặp, cho phép tối đa 150 kí tự, tự động loại bỏ những kí tự khoảng trắng ở phần đầu và phần cuối của tên,	String
2	Bắt đầu lúc	Có	Thời gian dự kiến bắt đầu cuộc gặp, chỉ cho phép lựa chọn thời gian trong tương lai, chặn thời gian sau thời gian kết thúc cuộc gặp	Date
3	Kết thúc lúc	Có	Thời gian dự kiến kết thúc cuộc gặp, chỉ cho phép lựa chọn thời gian trong tương lai, chặn thời gian trước thời gian bắt đầu cuộc gặp	Date

Bảng 2.2: Các trường trong phần tạo cuộc gặp

2.3.2 Đặc tả chức năng xem danh sách VC của bản thân

Mã ca sử dụng	UC02
Tên ca sử dụng	Xem danh sách VC của bản thân
Tên tác nhân	Người dùng
Mô tả	Ca sử dụng mô tả cách người dùng xem danh sách VC của bản thân
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống và được phân quyền sử dụng module VC
Luồng sự kiện chính	1. Chọn thẻ "Quản lý cuộc gặp của bản thân" 2. Nhấn "Danh sách cuộc họp của bạn" 3. Chọn một VC mà bạn muốn tham gia, hộp thoại chứa thông tin của VC hiển thị ra 4. Chọn "Tham gia ngay" để tham gia vào VC
Hậu điều kiện	Không có
Luồng sự kiện thay thế	4.1. Để mời người khác tham gia vào cuộc họp, chuyển sang use case "Mời người dùng khác tham gia vào VC" 4.2. Để cập nhật thông tin VC, nhấn vào "Cập nhật thông tin cuộc gặp", chuyển sang use case "Sửa thông tin cuộc gặp của bản thân" 4.3. Để xóa VC, nhấn vào "Xóa cuộc gặp", chuyển sang use case "Xóa một cuộc gặp đã lên lịch của bản thân" 4.4. Để mời người dùng khác tham gia vào VC của bản thân chọn chức năng "Mời bạn", chuyển sang use case UC05

Bảng 2.3: Đặc tả chức năng xem danh sách VC của bản thân

2.3.3 Đặc tả chức năng chia sẻ camera, micro

Mã ca sử dụng	UC03
Tên ca sử dụng	Chia sẻ camera, micro
Tên tác nhân	Người dùng
Mô tả	Ca sử dụng mô tả cách người dùng chia sẻ camera hoặc micro của bản thân cho những người dùng trong cùng một VC
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống, được phân quyền sử dụng module VC và hiện đang tham gia một VC bất kì
Luồng sự kiện chính	1. Nhấn vào biểu tượng camera để chia sẻ camera hoặc nhấn vào biểu tượng micro để chia sẻ micro 2. Hệ thống kiểm tra xem bạn đã cấp quyền sử dụng thiết bị tương ứng hay chưa 3. Yêu cầu cấp quyền sử dụng thiết bị tương ứng 4. Chọn "Allow" (Cấp quyền) để cấp quyền sử dụng thiết bị tương ứng cho hệ thống 5. Hệ thống hiển thị hình ảnh lấy từ camera lên màn hình của bạn và chia sẻ nó cho những người trong cùng VC hoặc phát âm thanh lên từ micro của bạn cho những người trong cùng VC
Hậu điều kiện	Không có
Luồng sự kiện thay thế	3.1. Nếu bạn đã cấp quyền sử dụng thiết bị tương ứng cho hệ thống từ trước, bỏ qua bước 3 và 4 4.1. Chọn "Block" nếu bạn muốn từ chối cấp quyền sử dụng thiết bị tương ứng cho hệ thống 4.1.1. Hệ thống thông báo cần được cấp quyền micro và camera 4.1.1.1. Nhấn "OK", micro hoặc camera của bạn sẽ không được sử dụng và không được chia sẻ cho người khác

Bảng 2.4: Đặc tả chức năng chia sẻ camera, micro

2.3.4 Đặc tả chức năng chia sẻ màn hình

Mã ca sử dụng	UC04
Tên ca sử dụng	Chia sẻ màn hình
Tên tác nhân	Người dùng
Mô tả	Ca sử dụng mô tả cách người dùng chia sẻ màn hình hoặc một cửa sổ làm việc hoặc một tab trong trình duyệt của bản thân cho những người dùng trong cùng một VC
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống, được phân quyền sử dụng module VC và hiện đang tham gia một VC bất kì
Luồng sự kiện chính	1. Nhấn vào biểu tượng có hình là chiếc màn hình máy tính 2. Hệ thống hiển thị hộp thoại "Choose what to share" 3. Lựa chọn màn hình mà bạn muốn chia sẻ 4. Nhấn "Share" 5. Hệ thống ghi lại phần màn hình mà bạn muốn chia sẻ, lấy nó để hiển thị lên màn hình của bạn và chia sẻ nó cho những người dùng trong cùng VC
Hậu điều kiện	Không có
Luồng sự kiện thay thế	3.1. Chọn thẻ "Window" để chia sẻ một cửa sổ làm việc 3.1.1 Chọn một cửa sổ làm việc mà bạn muốn chia sẻ 3.2 Chọn thẻ "Tab" để chia sẻ một tab trong trình duyệt 3.2.1 Chọn một tab mà bạn muốn chia sẻ

Bảng 2.5: Đặc tả chức năng chia sẻ màn hình

2.3.5 Đặc tả chức năng mời người dùng khác tham gia vào VC

Mã ca sử dụng	UC05
Tên ca sử dụng	Mời người dùng khác tham gia vào VC
Tên tác nhân	Người dùng
Mô tả	Ca sử dụng mô tả cách người dùng mời một người dùng khác tham gia vào VC hiện tại
Tiền điều kiện	Người dùng đã đăng nhập vào hệ thống và được phân quyền sử dụng module VC
Luồng sự kiện chính	1. Nhập tên người dùng mà bạn muốn mời 2. Hệ thống hiển thị danh sách những người dùng có tên chứa phần kí tự mà bạn đã nhập mà chưa được mời tham gia vào VC này từ trước 3. Chọn người dùng mà bạn muốn mời 4. Nhấn Enter hoặc chọn nút "Mời" để mời người dùng 5. Hệ thống hiển thị tin nhắn thành công nếu mời thành công.
Hậu điều kiện	Không có
Luồng sự kiện thay thế	5.1. Hệ thống hiển thị tin nhắn thất bại nếu mời thất bại.

Bảng 2.6: Đặc tả chức năng mời người dùng khác tham gia vào VC**2.4 Yêu cầu phi chức năng**

Đây là hệ thống Video conference nên yêu cầu phi chức năng quan trọng nhất là độ trễ (khoảng cách về thời gian tính từ lúc người dùng nói hoặc cử động hoặc gửi tin nhắn cho đến khi những người trong cùng VC nhận được) khi truyền nhận luồng tín hiệu âm thanh micro, hình ảnh camera, màn hình mà người dùng trong VC chia sẻ. Để đảm bảo chất lượng VC, trải nghiệm của người dùng, độ trễ phải nhỏ hơn 1 giây. Hình ảnh, âm thanh sắc nét (hình ảnh tối thiểu đủ để nhìn rõ khuôn mặt của đối phương, âm thanh không bị vọng, không chứa nhiều tạp âm). Sử dụng giao thức HTTPS, WSS để đảm bảo tính bảo mật cho hệ thống, luồng dữ liệu âm thanh, hình ảnh mà người dùng chia sẻ. Ngoài ra, hệ thống cần đảm bảo tính đồng nhất trong giao diện (màu sắc, kích thước các nút bấm, phông chữ, kích thước chữ cùng loại), sử dụng ngôn ngữ tiếng Việt do người dùng chính là người Việt.

CHƯƠNG 3. CÔNG NGHỆ SỬ DỤNG

Hệ thống Video Conference được xây dựng là một phân hệ trong dự án mã nguồn mở ERP [6] nên một số công nghệ được sử dụng trong hệ thống là được kế thừa từ dự án trên. Ở chương này, chúng tôi sẽ chia làm 2 phần. Phần một trình bày về những công nghệ đã sử dụng được kế thừa từ dự án ERP. Phần hai chúng tôi trình bày về những công nghệ mới mà chúng tôi đã tìm hiểu và áp dụng trong hệ thống Video Conference.

3.1 Công nghệ kế thừa

3.1.1 Java

Java [9] là một trong những ngôn ngữ lập trình phổ biến nhất trên thế giới hiện nay. Sau hơn hai thập kỉ tồn tại và phát triển, những đặc điểm hấp dẫn của nó cùng sự tiến hóa không ngừng đã giúp Java giữ vững vị trí quan trọng của mình trong cộng đồng lập trình viên toàn cầu. Có thể kể đến những điểm mạnh của Java như sau:

(i) Java là nền tảng độc lập, hoạt động trên máy ảo JVM vì thế có thể chạy mã Java trên bất kì loại máy tính Mac, Windows, Linux, từ đó mà mã nguồn viết bằng Java có thể dễ dàng triển khai cũng như bảo trì.

(ii) Java là ngôn ngữ lập trình hướng đối tượng, hỗ trợ đầy đủ các tính chất của hướng đối tượng như trừu tượng hóa, đóng gói, kế thừa, đa hình.

(iii) Quản lý bộ nhớ, dọn dẹp các ngăn nhớ không được sử dụng một cách tự động, ngăn chặn các phương thức tấn công liên quan đến bộ nhớ.

3.1.2 Springboot

Spring-boot [10] là framework của Java được giới thiệu vào năm 2012 và vẫn đang được phát triển từ đó đến nay. Một trong những điểm mạnh của Spring-boot là:

(i) Hỗ trợ mô hình MVC giúp cho lập trình viên dễ dàng xây dựng và phát triển một ứng dụng web.

(ii) Tomcat engine được cấu hình sẵn trong Spring-boot nên lập trình viên không cần phải cấu hình lại.

(iii) Hỗ trợ tốt truy vấn cơ sở dữ liệu quan hệ như Mysql, Postgres, Oracle.

3.1.3 ReactJs

ReactJs là thư viện của JavaScript được Facebook phát triển và giới thiệu vào năm 2012, là một trong những Framework hỗ trợ lập trình giao diện web phổ biến nhất hiện nay. React làm cho việc tạo giao diện người dùng tương tác trở nên dễ dàng hơn. Thiết kế các khung nhìn đơn giản cho từng trạng thái trong ứng dụng. React sẽ cập nhật, hiển thị các thành phần phù hợp một cách hiệu quả khi dữ liệu thay đổi [11].

Một tính năng mới của React được ra mắt tại phiên bản 16.8 là Hooks. Chúng cho phép lập trình viên sử dụng các tính năng của React mà không cần viết một lớp. [12]

3.1.4 PostgreSQL

PostgreSQL là một hệ thống cơ sở dữ liệu quan hệ đối tượng mã nguồn mở mạnh mẽ, sử dụng và mở rộng ngôn ngữ SQL kết hợp với nhiều tính năng giúp lưu trữ và chia tỷ lệ một cách an toàn các khối lượng công việc dữ liệu phức tạp nhất. Nguồn gốc của PostgreSQL có từ năm 1986 như một phần của dự án POSTGRES tại Đại học California ở Berkeley và đã có hơn 30 năm phát triển tích cực trên nền tảng cốt lõi.

PostgreSQL đã tạo được danh tiếng mạnh mẽ về kiến trúc đã được chứng minh, độ tin cậy, tính toàn vẹn của dữ liệu, bộ tính năng mạnh mẽ, khả năng mở rộng và sự cống hiến của cộng đồng nguồn mở đằng sau phần mềm để liên tục cung cấp các giải pháp hiệu quả và sáng tạo. PostgreSQL chạy trên tất cả các hệ điều hành chính, đã tuân thủ ACID từ năm 2001 và có các tiện ích bổ sung mạnh mẽ như bộ mở rộng cơ sở dữ liệu không gian địa lý PostGIS phổ biến. Không có gì ngạc nhiên khi PostgreSQL đã trở thành cơ sở dữ liệu quan hệ nguồn mở được nhiều người và tổ chức lựa chọn [13].

3.2 Công nghệ đóng góp

3.2.1 WebSocket

WebSocket là công nghệ hỗ trợ giao tiếp hai chiều (full-duplex) giữa client và server bằng cách sử dụng một TCP socket để tạo một kết nối hiệu quả và ít tốn kém. Giao thức WebSocket là một giao thức được sử dụng rộng rãi cho việc phát triển ứng dụng đòi hỏi tính real-time (thời gian thực, tốc độ phản hồi nhanh). WebSocket được áp dụng trong hệ thống Video Conference để đảm bảo tính thời gian thực khi nhắn tin, chia sẻ hình ảnh, âm thanh.

Ngoài WebSocket, những phương thức trước đó để mô phỏng kết nối full-duplex được dựa trên polling, một phương thức đồng bộ mà client gửi request đến sever để

xem có thông tin không. Client nhận response từ server cả khi mà không có thông tin có sẵn. Polling chỉ hoạt động tốt khi khoảng thời gian message có sẵn có thể biết trước được. Tuy nhiên, việc người dùng khi nào chia sẻ âm thanh, hình ảnh, nhắn tin là không thể dự đoán được. Thêm vào đó polling khiến client mở và đóng nhiều kết nối không cần thiết.

Long polling (hay Comet) là một phương thức giao tiếp phổ biến mà trong đó client mở kết nối đến server trong một giai đoạn nhất định. Nếu server không có thông tin gì, nó sẽ giữ kết nối đến khi có thông tin để trả về cho client hoặc đến khi đạt giới hạn thời gian cho kết nối đó. Về bản chất, Long polling trì hoãn sự hoàn thành của HTTP response cho đến khi server có gì đó để trả về client, kỹ thuật này thường được gọi là hanging-GET hay pending- POST. Thực tế là việc client phải thường xuyên reconnect đến server khiến long polling không phải là một sự lựa chọn tối ưu cho các ứng dụng real-time nói chung và hệ thống Video Conference nói riêng [14].

3.2.2 WebRTC

WebRTC (Web Real-Time Communication) là một tập hợp các hàm lập trình (web API) được phát triển bởi World Wide Web Consortium (W3C). WebRTC hỗ trợ trình duyệt giao tiếp với nhau theo thời gian thực bằng video, âm thanh hay truyền tải dữ liệu "Peer-to-peer" (P2P) mà trình duyệt không cần cài đặt thêm bất kỳ một công cụ hỗ trợ bên ngoài nào khác. Chi tiết về công nghệ sẽ được trình bày trong phần 5.2

3.2.3 NodeJs

NodeJs [15] là một nền tảng được xây dựng trên "V8 Javascript engine" được viết bằng C++ và Javascript. Nền tảng này được phát triển bởi Ryan Lienhart Dahl vào năm 2009. Một số điểm mạnh có thể kể đến khi nói về NodeJs như sử dụng mô hình non-blocking I/O làm cho nó nhẹ và hiệu quả hơn, hệ thống thư viện của NodeJs, npm là hệ thống thư viện mã nguồn mở lớn nhất thế giới. Hệ thống Video Conference cũng đã sử dụng một số thư viện của npm như PeerJs và PeerServer để giải quyết bài toán kết nối peer-to-peer giữa những người dùng.

3.2.4 PeerJs và PeerServer

PeerJs và PeerServer là hai thư viện Javascript mã nguồn mở thuộc hệ thống thư viện npm. PeerServer cho phép lập trình viên tự tạo một TURN server bằng nodeJS, cung cấp cho mỗi người dùng một mã định danh peerId duy nhất. Từ đó thông qua PeerJs và mã định danh của nhau, một người dùng bất kỳ có thể kết nối P2P để truyền nhận dữ liệu, luồng hình ảnh và âm thanh với người còn lại. Chức năng của 2 thư viện này sẽ được nêu rõ ở phần 5.3.

CHƯƠNG 4. THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Mô hình ba tầng (Three-tier architecture) là một kiến trúc ứng dụng phần mềm tổ chức các ứng dụng thành ba tầng tính toán logic và vật lý: tầng trình bày (presentation tier), tầng ứng dụng (application tier) và tầng dữ liệu (data tier). Lợi ích chính của kiến trúc ba tầng là mỗi tầng chạy trên một cơ sở hạ tầng riêng, được phát triển đồng thời bởi một nhóm phát triển riêng biệt và có thể được cập nhật hoặc mở rộng mà không ảnh hưởng đến các tầng khác [16].



Hình 4.1: Mô hình ba tầng

a, Tầng trình bày

Tầng trình bày là tầng giao diện người dùng và giao tiếp của ứng dụng, nơi người dùng cuối (end-user) tương tác với ứng dụng. Mục đích chính của nó là hiển thị thông tin và thu thập thông tin từ người dùng. Tầng cấp cao nhất này có thể chạy trên trình duyệt web, dưới dạng ứng dụng dành cho máy tính để bàn hoặc giao diện người dùng đồ họa (GUI) chẳng hạn. Các tầng trình bày web thường được phát triển bằng HTML, CSS và JavaScript. Các ứng dụng máy tính để bàn có thể được viết bằng nhiều ngôn ngữ khác nhau tùy thuộc vào nền tảng.

b, Tầng ứng dụng

Tầng ứng dụng, còn được gọi là tầng logic hoặc tầng giữa, là trung tâm của hệ thống. Trong tầng này, thông tin được thu thập từ tầng trình bày được xử lý, đôi khi không giống hoặc không tồn tại trong những thông tin được lưu trong tầng dữ liệu. Bằng cách sử dụng logic nghiệp vụ, bộ quy tắc nghiệp vụ cụ thể, tầng ứng dụng có thể thêm, xóa hoặc sửa đổi dữ liệu trong tầng dữ liệu. Tầng ứng dụng thường được phát triển bằng Python, Java, Perl, PHP,...

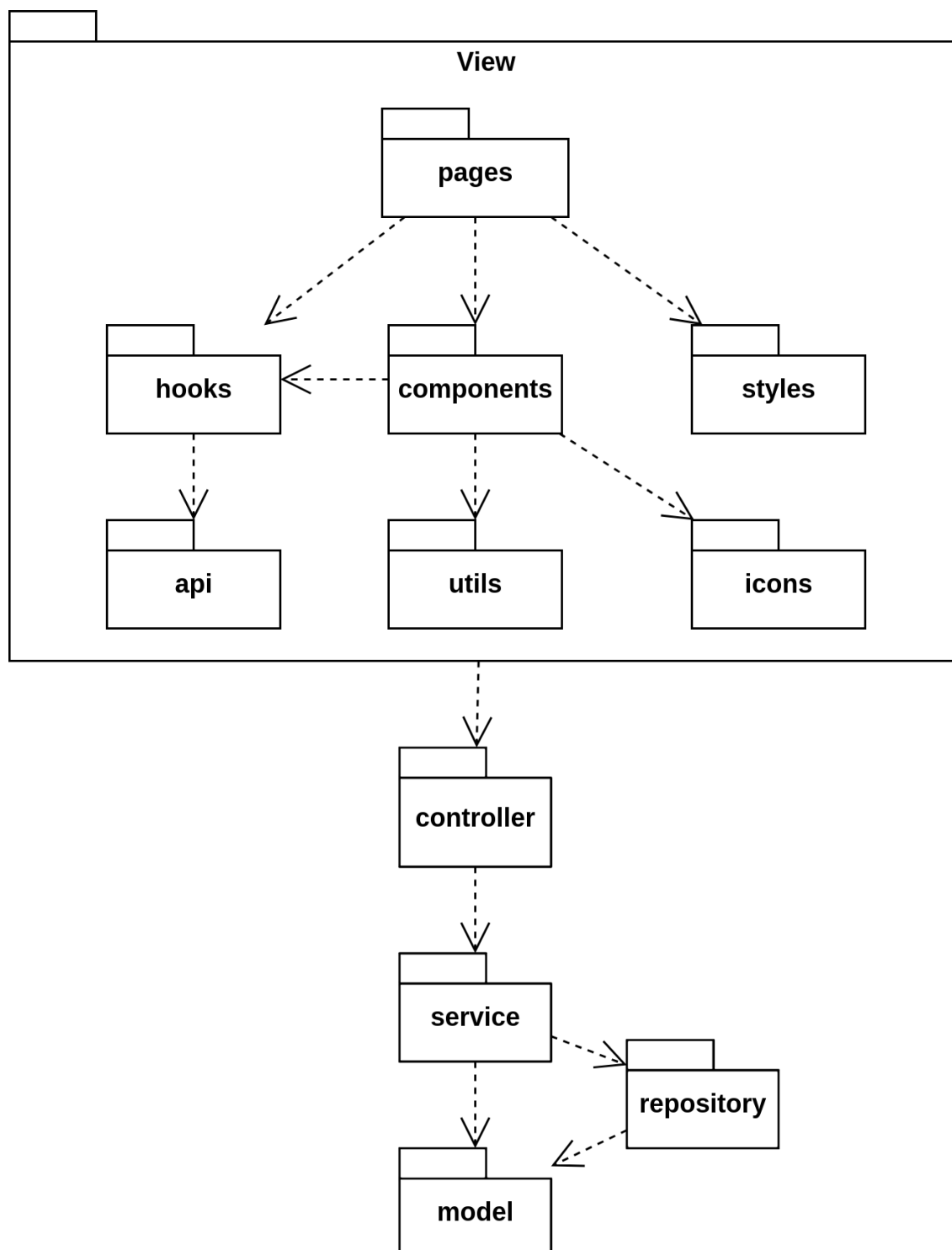
c, Tầng dữ liệu

Tầng dữ liệu, đôi khi được gọi là tầng cơ sở dữ liệu, tầng truy cập dữ liệu hoặc tầng phụ, là nơi lưu trữ và quản lý thông tin được ứng dụng xử lý. Các hệ thống cơ sở dữ liệu phổ biến có thể kể đến như PostgreSQL, MySQL, MariaDB, DB2,

Informix, Microsoft SQL Server hoặc MongoDB.

Tại hệ thống Video Conference, tầng trình bày được phát triển bằng HTML, CSS và Javascript thông qua ReactJs, tầng ứng dụng được phát triển bằng Java và framework Spring-boot, tầng dữ liệu sử dụng cơ sở dữ liệu PostgreSQL.

4.1.2 Thiết kế tổng quan



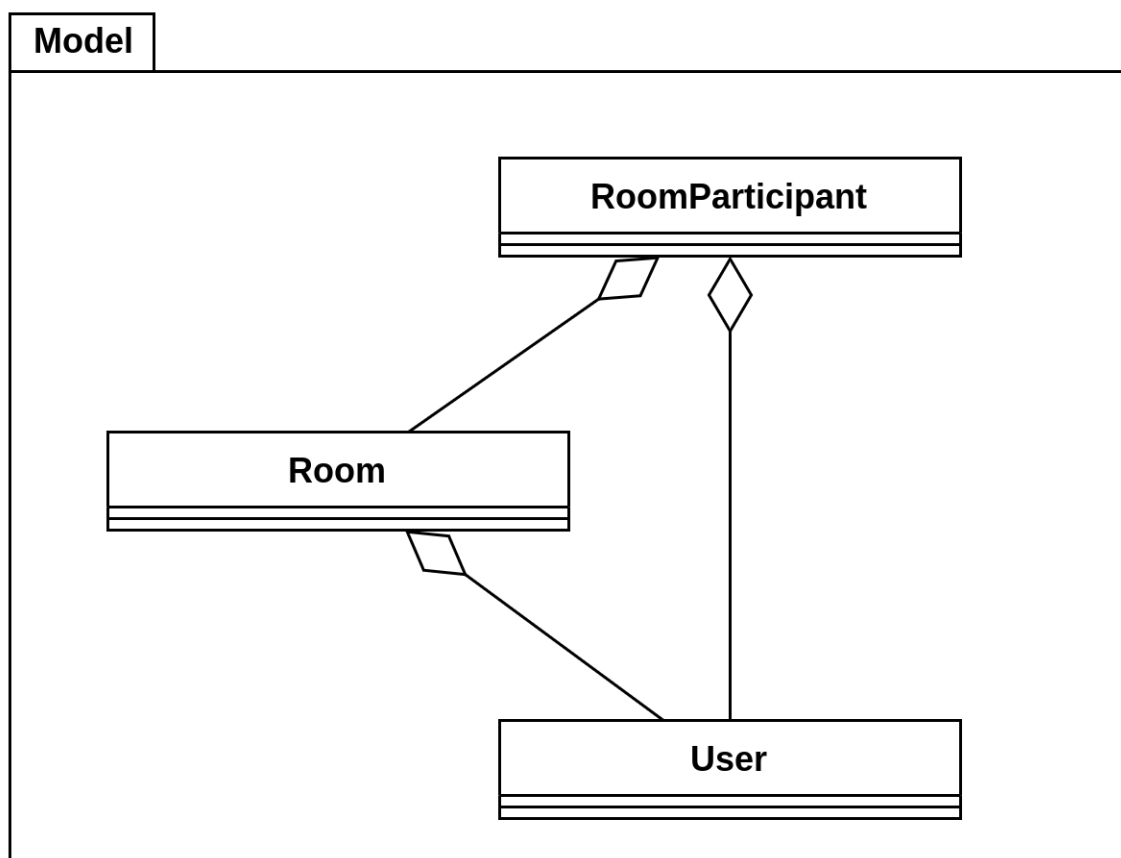
Hình 4.2: Biểu đồ phụ thuộc gói

Hình 4.2 là biểu đồ phụ thuộc gói của hệ thống, với mục đích/nhiệm vụ của từng gói được nêu ở bảng 4.1.

STT	Tên gói	Mục đích/Nhiệm vụ
1	View	Gồm các thành phần tạo nên những giao diện trực quan giúp người dùng tương tác và sử dụng hệ thống. Trong gói View gồm các gói con pages, api, components, styles, hooks, utils, icons.
2	Pages	Là thiết kế giao diện của các màn hình trong hệ thống. Mỗi một class trong gói tương ứng với một màn hình của hệ thống.
3	Components	Là những thành phần nhỏ giúp hiển thị giao diện hoặc xử lý logic cấu thành nên một page.
4	Styles	Là tập hợp các tập tin giúp cải thiện chất lượng giao diện, làm đẹp hơn, đảm bảo các tính chất khi thiết kế giao diện và trải nghiệm người dùng trên các màn hình của hệ thống.
5	Hooks	Tập hợp các hook tự phát triển (Hook - được định nghĩa và giải thích trong phần 3.1.3).
6	Api	Thực hiện các yêu cầu xử lý, truy vấn dữ liệu của client đến server. Nhận phản hồi từ server và xử lý.
7	Utils	Tập hợp các tiện ích, các hằng số được sử dụng trong các component.
8	Icons	Tập hợp các icon được sử dụng trong các component.
9	Controller	Nhận các yêu cầu từ Client gửi đến, phân quyền, xác thực danh tính và gửi xuống Service để xử lý.
10	Service	Gồm các lớp xử lý logic các yêu cầu từ controller gửi xuống, sử dụng các phương thức được cung cấp bởi repository để truy vấn đến cơ sở dữ liệu.
11	Repository	Gồm các class cung cấp những phương thức thực hiện các câu truy vấn đến cơ sở dữ liệu.
12	Model	Tập hợp các lớp định nghĩa thông tin cơ bản của các đối tượng trong hệ thống, mỗi class tương ứng với một bảng trong database.

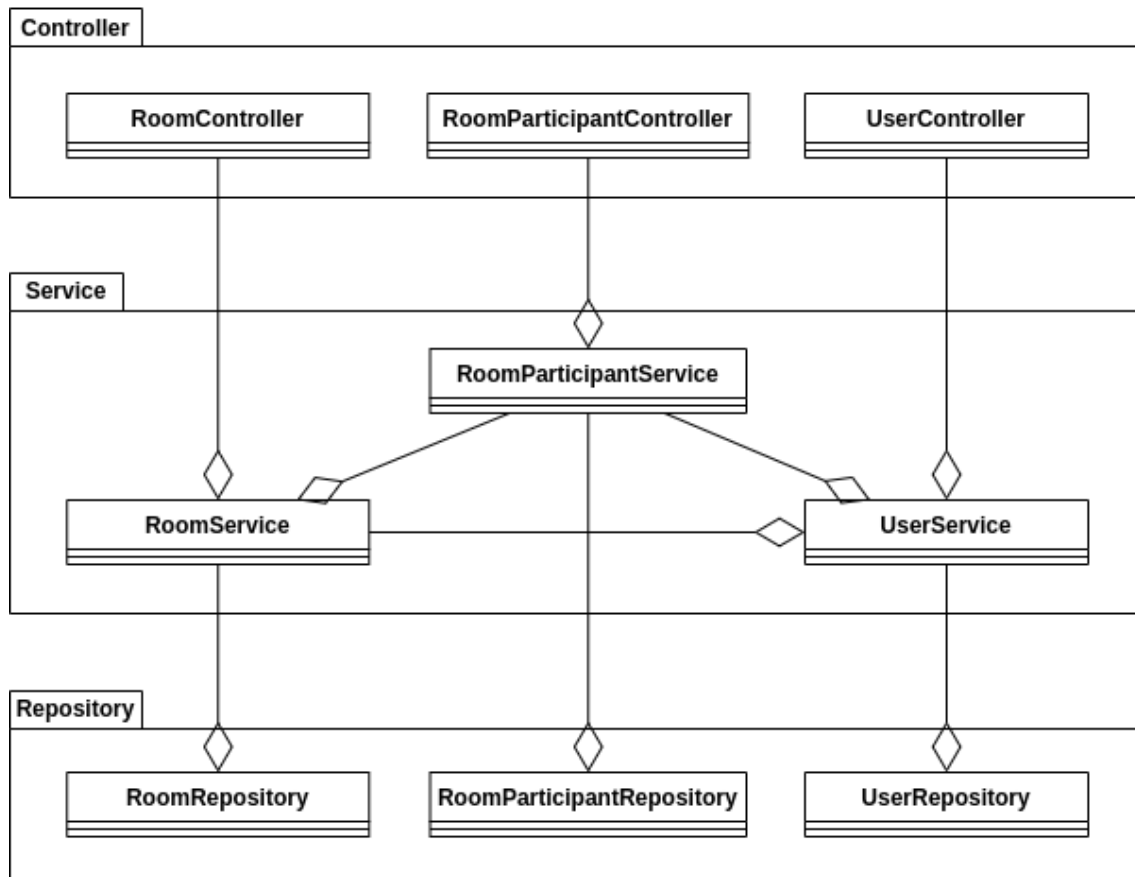
Bảng 4.1: Mục đích/Nhiệm vụ của các gói trong hệ thống

4.1.3 Thiết kế chi tiết gói



Hình 4.3: Biểu đồ thiết kế gói Model

Hình 4.3 là biểu đồ thiết kế cho gói Model. Gói Model tập hợp các lớp định nghĩa thông tin cơ bản của các đối tượng trong hệ thống bao gồm User - người dùng, Room - cuộc gặp, RoomParticipant - người tham gia cuộc gặp. RoomParticipant có quan hệ kết tập với User và Room vì mỗi đối tượng RoomParticipant bao gồm thông tin đó là User nào và tham gia vào Room nào. Ngoài ra, mỗi Room lại có một chủ sở hữu (người tạo Room) nên Room có quan hệ kết tập với User.



Hình 4.4: Biểu đồ thiết kế gói Controller, Service, Repository

Hình 4.4 là biểu đồ thiết kế cho gói Controller, Service, Repository. Các lớp trong gói Controller có nhiệm vụ là nhận yêu cầu từ client sau đó chuyển xuống cho các lớp Service tương ứng của nó xử lý. Ví dụ lớp RoomController có một thuộc tính là một đối tượng RoomService, khi nhận được yêu cầu từ client, RoomController sẽ sử dụng các phương thức được cung cấp bởi RoomService để xử lý yêu cầu đó. Các lớp trong gói Service có nhiệm vụ xử lý logic. Lớp UserService đảm nhận vai trò xử lý logic với những yêu cầu liên quan đến người dùng. Để thao tác với dữ liệu về người dùng trong cơ sở dữ liệu, nó sẽ sử dụng những phương thức được cung cấp bởi UserRepository. Tương tự, RoomService đảm nhận vai trò xử lý logic với những yêu cầu liên quan đến cuộc gặp. Để thao tác với dữ liệu về cuộc gặp trong cơ sở dữ liệu, nó gọi đến những phương thức được cung cấp bởi RoomRepository, ngoài ra để truy xuất dữ liệu liên quan đến người dùng, RoomService sẽ gọi đến các phương thức mà UserService cung cấp. Cuối cùng, RoomParticipantService đảm nhận vai trò xử lý logic với những yêu cầu liên quan đến người tham gia cuộc gặp. Nó sử dụng các phương thức do RoomParticipantRepository, RoomService và UserService cung cấp. Thiết kế này đảm bảo rằng, tất cả các lớp Controller sẽ chỉ có một nhiệm vụ duy nhất là nhận yêu cầu từ client, việc xử lý logic sẽ do lớp Service của chính nó đảm nhận, các đối tượng Repository chỉ cung

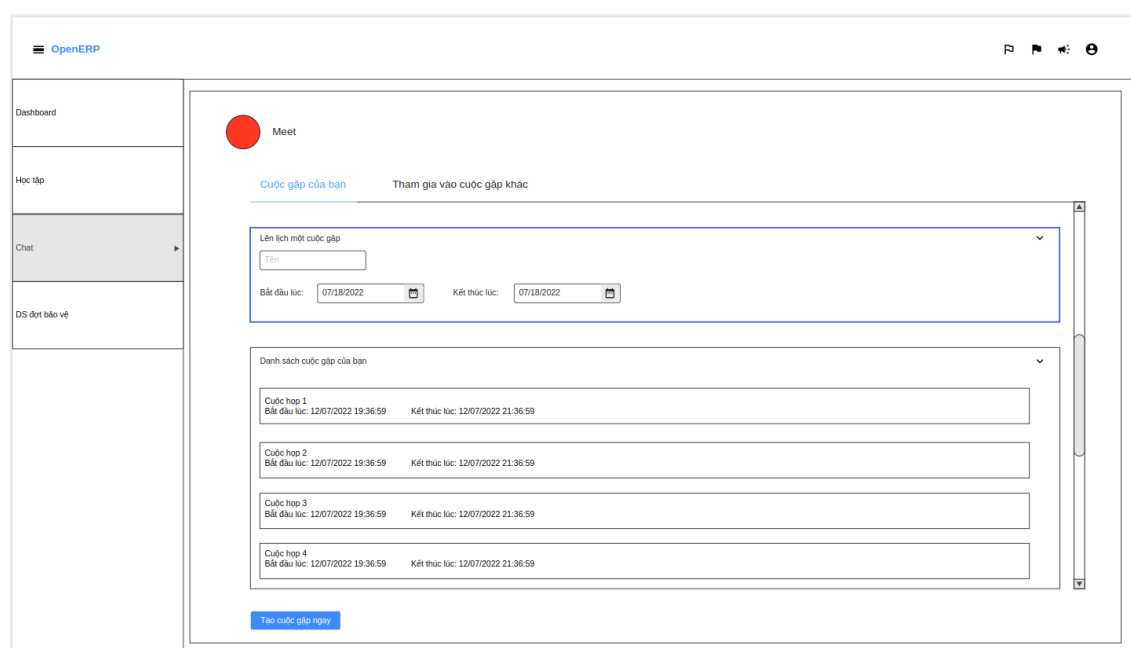
cấp các phương thức cho chính Service của nó. Để thao tác với dữ liệu, Service sẽ gọi tới Repository của chính nó hoặc sử dụng phương thức do Service khác cung cấp.

4.2 Thiết kế chi tiết

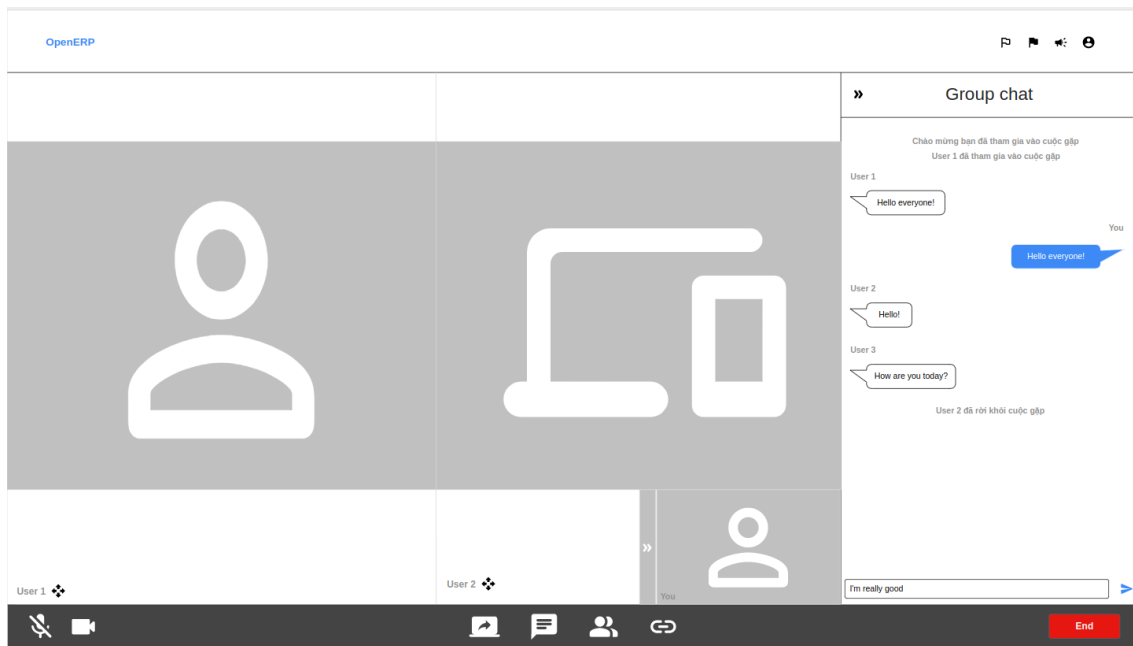
4.2.1 Thiết kế giao diện

Hệ thống sẽ hoạt động tốt trên các thiết bị có phần cứng mạnh mẽ, vì vậy chỉ hỗ trợ các thiết bị máy tính để bàn hoặc máy tính xách tay với kích thước màn hình từ 14 inch trở lên. Người dùng nên sử dụng màn hình có độ phân giải 1440 x 900 pixels để có trải nghiệm về hình ảnh tốt nhất. Thông điệp phản hồi của hệ thống sẽ được đặt ở góc trái dưới, với hai thông tin chính là tiêu đề và nội dung chính của thông điệp, những thông báo thành công sẽ có màu xanh lá, thông báo thất bại sẽ có màu đỏ. Màu chủ đạo của hệ thống là màu xanh dương. Các nút được thiết kế đồng nhất với màu mặc định là màu xanh dương, chiều cao 36px, không viền, chữ màu trắng, border-radius 4px.

Một số hình ảnh minh họa thiết kế giao diện của hệ thống.



Hình 4.5: Thiết kế màn hình trang chủ



Hình 4.6: Thiết kế màn hình cuộc gặp

4.2.2 Thiết kế lớp

Dưới đây là một số thiết kế lớp và biểu đồ trình tự minh họa một số use case đặc trưng của hệ thống.

RoomParticipantController
- roomParticipantService: RoomParticipantService
+ handleMessage (Map<String, String> message, String roomId): Map<String, String> + getAllParticipantsInThisRoom (String roomId): ResponseEntity<?> + inviteParticipant (HashMap<String, String> roomParticipant): ResponseEntity<?> + getInvitedParticipants (String roomId): ResponseEntity<?> + searchUserByIdToInvite (String searchString): ResponseEntity<?> + getListInvitedRoom(String userId): ResponseEntity<?>

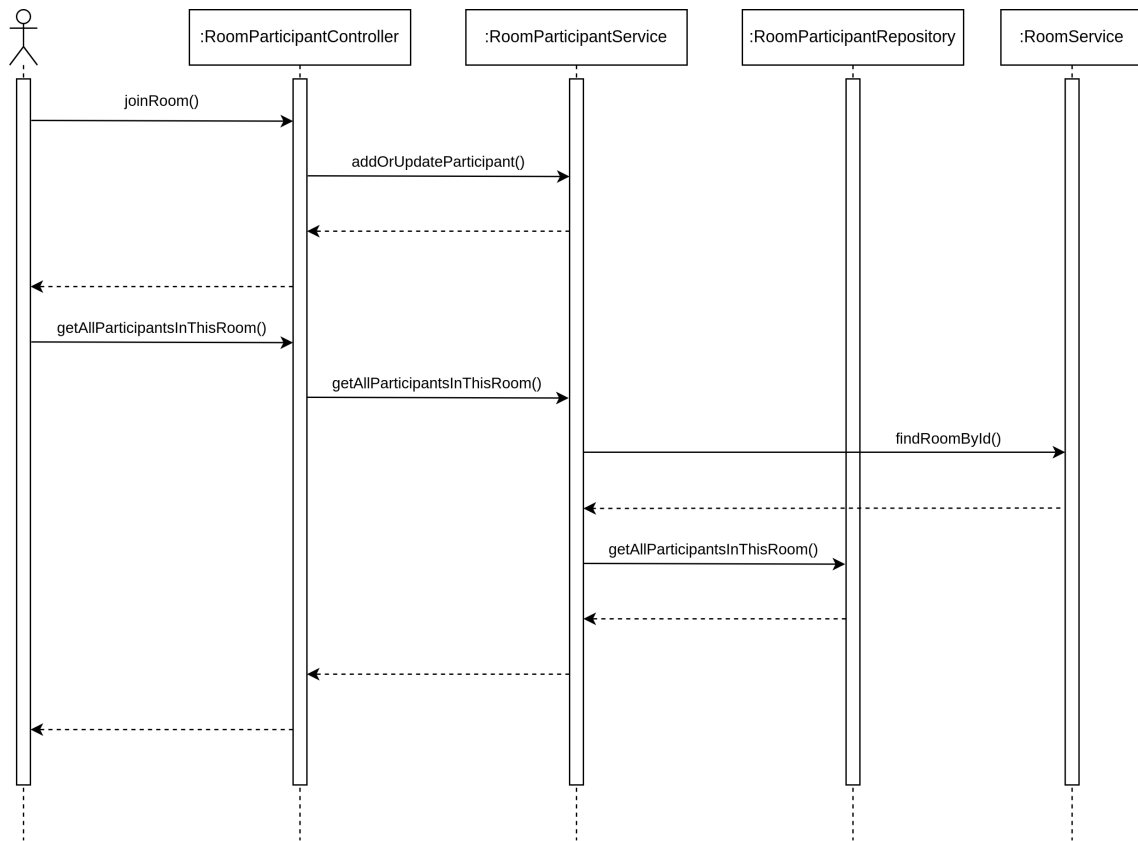
Hình 4.7: Thiết kế lớp RoomParticipantController

RoomParticipantService
- roomParticipantRepository: RoomParticipantRepository - roomService: RoomService - userService: UserService
+ addOrUpdateParticipant (String roomId, String userId, String peerId): String + outRoom (String roomId, String userId): void + getAllParticipantInThisRoom (String roomId): List<Map<String, String>> + inviteParticipant (UUID roomId, String userId): void + searchUsersById (Pageable page, String searchString, String roomId): Page<String> + getListInvitedRoom (Pageable page, String userId): Page<Room> + getInvitedParticipants (Pageable page, String roomId): Page<String>

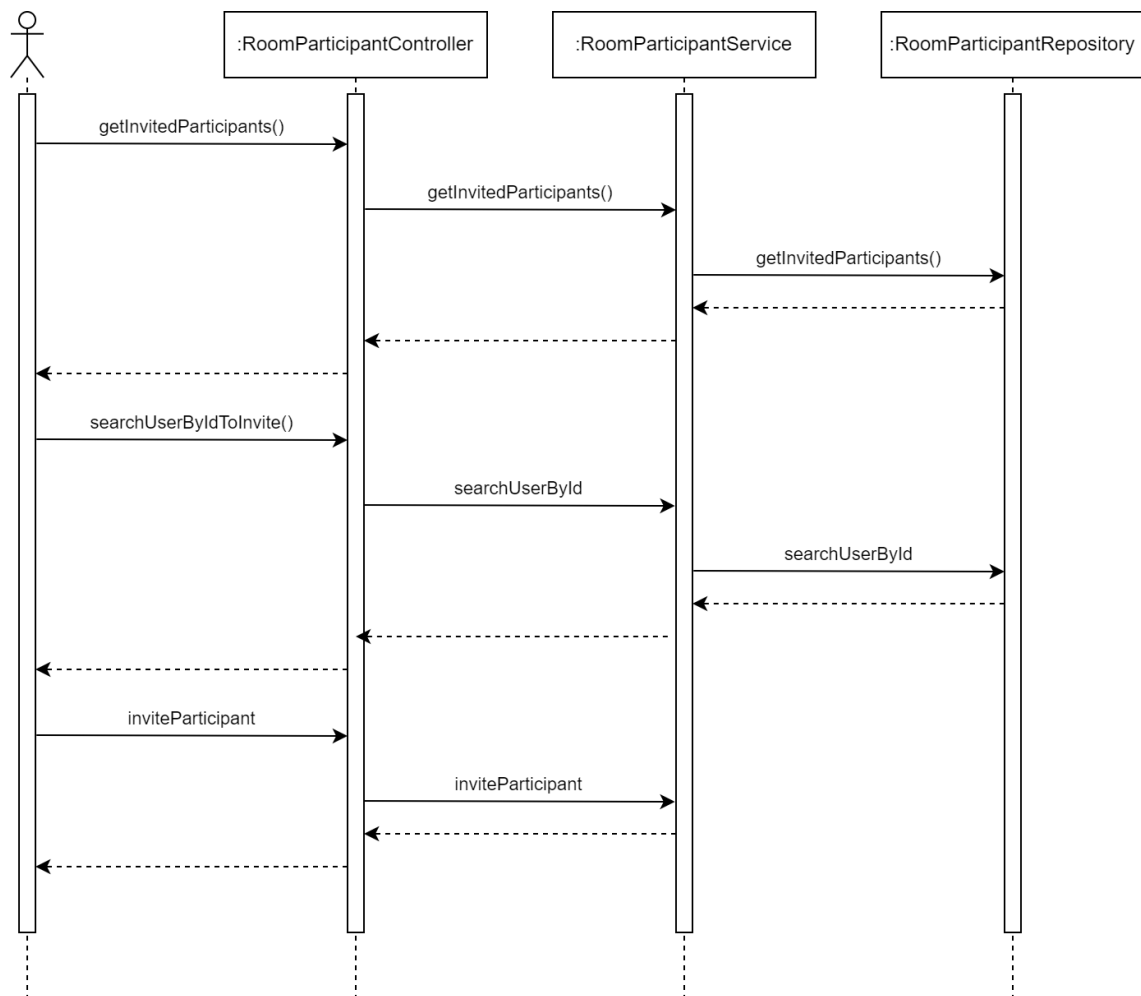
Hình 4.8: Thiết kế lớp RoomParticipantService

RoomParticipantRepository
+ getAllParticipantInThisRoom (Room room): List<Object[]> + outRoom (Room room, User participant): void + updatePeer (RoomParticipant roomParticipant, String peerId): void + findByParticipantAndRoom (User participant, Room room): Optional<RoomParticipant> + searchUsersById (Pageable page, String searchString, Room room): Page<String> + getListInvitedRoom (Pageable page, User user): Page<Room> + getInvitedParticipants (Pageable page, Room room): Page<String>

Hình 4.9: Thiết kế lớp RoomParticipantRepository

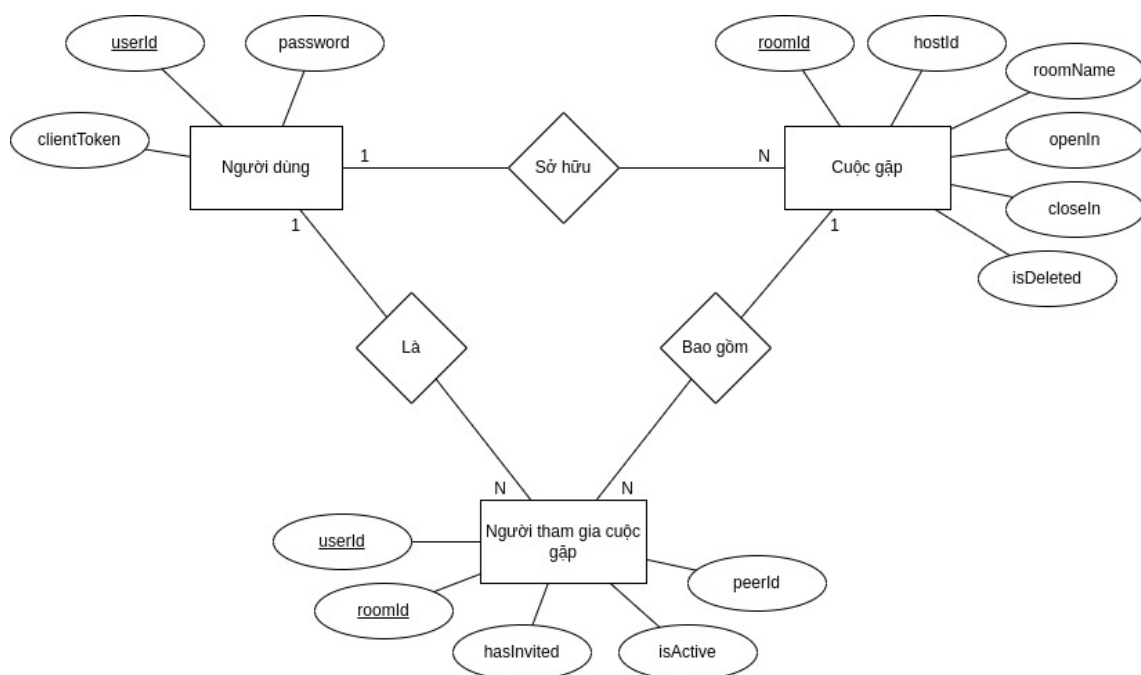


Hình 4.10: Biểu đồ trình tự tham gia một cuộc gặp



Hình 4.11: Biểu đồ trình tự mời người dùng khác tham gia vào cuộc gặp

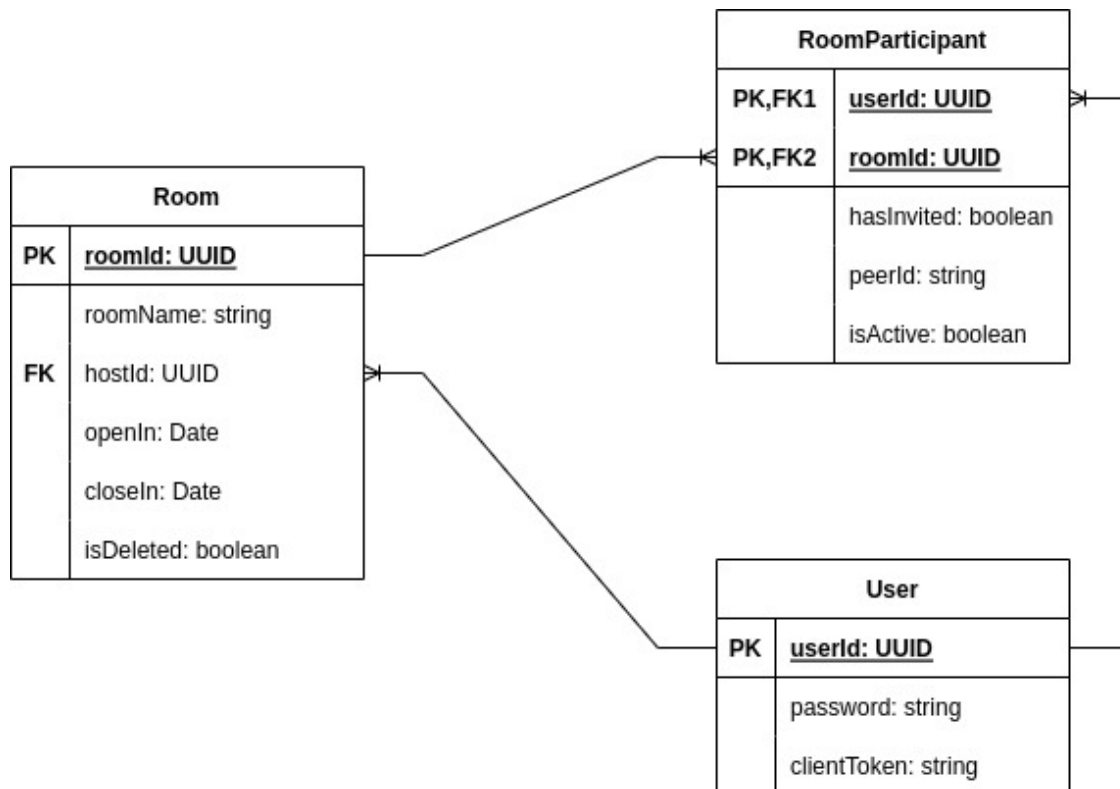
4.2.3 Thiết kế cơ sở dữ liệu



Hình 4.12: Biểu đồ thực thể liên kết (E-R Diagram)

Hình 4.12 là biểu đồ thực thể liên kết của hệ thống, gồm hai thực thể chính là người dùng và cuộc gặp. Mỗi người dùng có `userId` riêng biệt, `password` dùng để đăng nhập vào hệ thống và `clientToken` dùng để lưu trữ thông tin của người dùng đã được mã hóa để giảm thiểu số lần cần đăng nhập vào hệ thống. Một cuộc gặp bao gồm các thông tin `roomId` riêng biệt để phân biệt các cuộc gặp với nhau, `roomName` là tên của cuộc gặp, `openIn`, `closeIn` là thời gian bắt đầu vào kết thúc, `hostId` là `userId` của chủ sở hữu cuộc gặp, `isDeleted` là trường phân biệt xem cuộc gặp này đã bị xóa hay chưa. Một người dùng có thể là chủ sở hữu của nhiều cuộc gặp. Một cuộc gặp có một và chỉ một chủ sở hữu.

Ngoài ra, một người dùng có thể tham gia vào nhiều cuộc gặp, một cuộc gặp có thể bao gồm nhiều người dùng cùng tham gia. Vì thế chúng tôi tạo thêm một thực thể nữa là người tham gia cuộc gặp. Một người dùng có thể là nhiều người tham gia cuộc gặp tham gia vào nhiều cuộc gặp khác nhau. Một cuộc gặp bao gồm nhiều người tham gia cuộc gặp. Mỗi người tham gia cuộc gặp là một thể hiện của người dùng tham gia vào một cuộc gặp tương ứng. Người tham gia cuộc gặp có khóa chính là `userId` X và `roomId` Y lưu trữ thông tin người dùng X đang ở trong cuộc gặp Y, `isActive` thể hiện xem người dùng hiện có đang tham gia cuộc gặp hay không, `hasInvited` lưu trữ thông tin người dùng đã được mời tham vào cuộc gặp hay chưa và `peerId` là mã định danh peer để kết nối peer-to-peer dùng để chia sẻ micro, camera, màn hình với người dùng khác khi tham gia vào cuộc gặp.



Hình 4.13: Biểu đồ thiết kế cơ sở dữ liệu

Hình 4.13 là biểu đồ thiết kế cơ sở dữ liệu cho cơ sở dữ liệu SQL.

Các hình dưới đây biểu thị danh sách và ý nghĩa các bảng trong cơ sở dữ liệu và chi tiết các trường trong từng bảng.

STT	Bảng	Ý nghĩa
1	Room	Lưu thông tin về cuộc gặp
2	User	Lưu thông tin về người dùng
3	RoomParticipant	Lưu thông tin về người tham gia một cuộc gặp

Bảng 4.2: Danh sách và ý nghĩa các bảng trong cơ sở dữ liệu

STT	Tên trường	Kiểu dữ liệu	Mô tả
1	roomId	UUID	Mã định danh duy nhất của cuộc gặp
2	roomName	String	Tên của cuộc gặp
3	hostId	UUID	Mã định danh của chủ phòng
4	openIn	Date	Thời gian bắt đầu cuộc gặp
5	closeIn	Date	Thời gian kết thúc cuộc gặp
6	isDeleted	Boolean	Cờ đánh dấu xem cuộc gặp đã bị xóa hay chưa

Bảng 4.3: Bảng Room

STT	Tên trường	Kiểu dữ liệu	Mô tả
1	userId	UUID	Mã định danh duy nhất của người dùng
2	password	String	Mật khẩu của tài khoản người dùng
3	clientToken	String	Thông tin đã được mã hóa của người dùng

Bảng 4.4: Bảng User

STT	Tên trường	Kiểu dữ liệu	Mô tả
1	userId	UUID	Mã định danh duy nhất của người dùng
2	roomId	UUID	Mã định danh duy nhất của cuộc gặp
3	hasInvited	Boolean	Trạng thái được mời hay chưa của người tham gia
4	isActive	Boolean	Trạng thái hoạt động của người tham gia
5	peerId	String	Mã định danh peer của người tham gia

Bảng 4.5: Bảng RoomParticipant

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Bảng 4.6 dưới đây liệt kê những thư viện, công cụ mà chúng tôi sử dụng trong hệ thống.

Mục đích	Công cụ	Phiên bản	Địa chỉ URL
IDE lập trình	Visual Studio Code	1.69.1	https://code.visualstudio.com/
Ngôn ngữ lập trình	Javascript	1.7	https://www.javascript.com/
Ngôn ngữ lập trình	Java	1.8	https://www.java.com/en/
Framework Front-end	ReactJs	17.0.2	https://reactjs.org/
Framework Back-end	Springboot	2.7.2	https://spring.io/
Platform Back-end	NodeJs	18.0.0	https://nodejs.org/en/
Framework Back-end	Express	4.18.1	https://expressjs.com/
Trình quản lý thư viện Javascript	Npm	6.8.0	https://www.npmjs.com/
Trình quản lý thư viện Java	Maven	3.8.6	https://maven.apache.org/
Thư viện UI	Material UI	4.9.11	https://mui.com/
Thư viện HTTP Client	Axios	0.21.1	https://axios-http.com/
Thư viện quản lý server state	React Query	3.38.0	https://tanstack.com/query/v3
Thư viện Socket trên client	Socket.io-client	4.5.0	https://socket.io/
Thư viện peer-to-peer	Peerjs	1.3.2	https://peerjs.com/
Thư viện server cho peerjs	PeerServer	0.6.1	https://peerjs.com/peerserver

Bảng 4.6: Danh sách thư viện và công cụ sử dụng

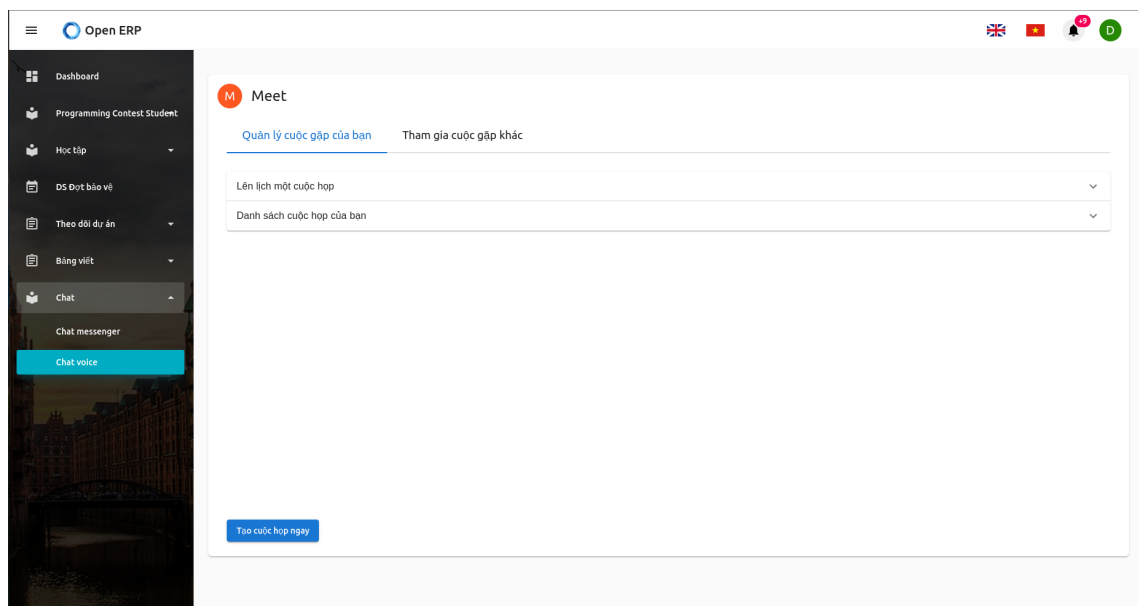
4.3.2 Kết quả đạt được

Hệ thống Video Conference đã được deploy, có thể sử dụng ở mọi nơi và là một module trong dự án ERP, người dùng có thể truy cập bằng đường dẫn `https://openerp.dailyopt.ai/chat/voice/main`. Sản phẩm đã được thử nghiệm và sử dụng bởi nội bộ nhóm phát triển. Những buổi báo cáo tiến độ DATN đã thực hiện trên chính hệ thống này. Sản phẩm đem lại trải nghiệm đơn giản hơn với người dùng của hệ thống ERP, giúp thầy và trò hiểu hơn về công nghệ WebRTC, kết nối P2P, video call. Bảng 4.7 thống kê một số thông tin của sản phẩm.

	Số lượng	Đơn vị
Số dòng code	3263	Dòng
Số lớp	46	
Số gói	12	
Dung lượng mã nguồn	185	KB

Bảng 4.7: Thống kê một số thông tin của sản phẩm

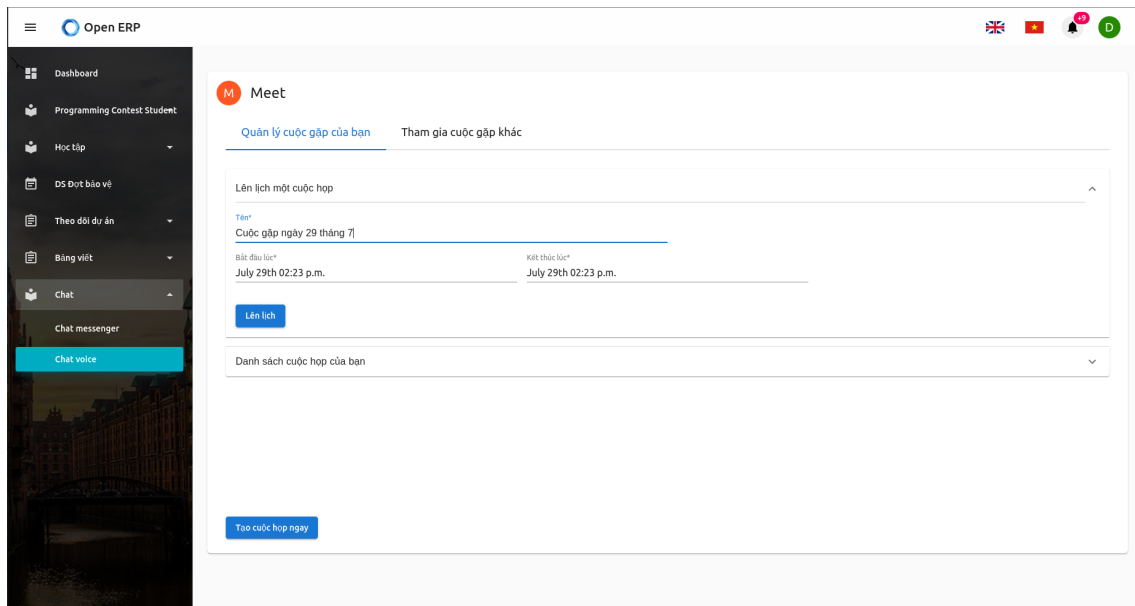
4.3.3 Minh họa các chức năng chính



Hình 4.14: Giao diện trang chủ

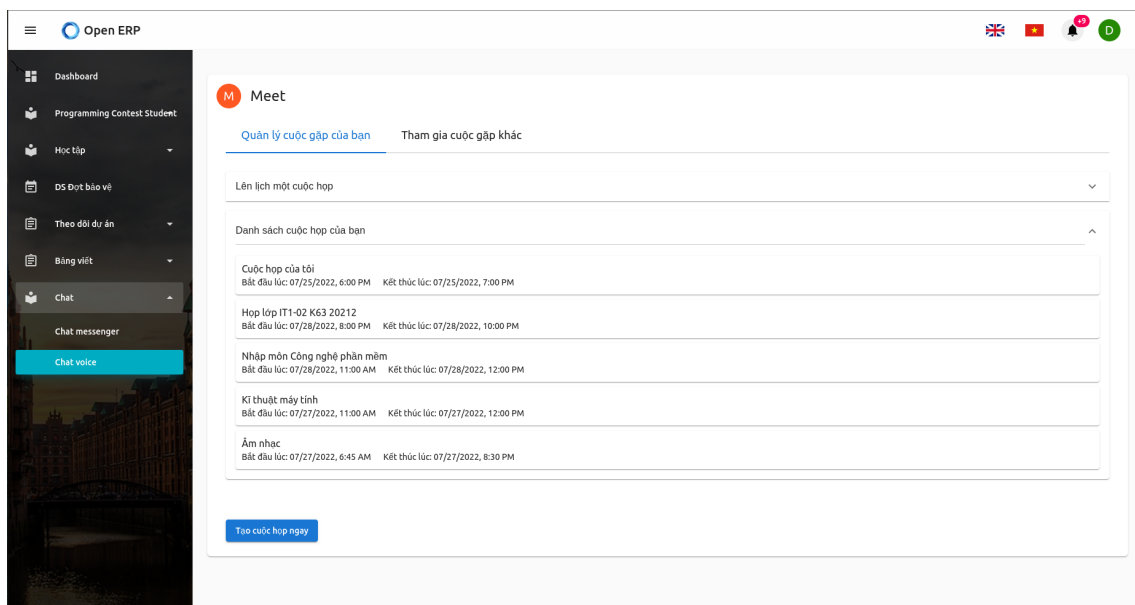
Hình 4.14 là giao diện của màn hình trang chủ. Phần bên trái được gọi là SideBar giúp người dùng chuyển nhanh sang các module khác trong dự án OpenERP. Phần bên phải là trang chủ của hệ thống. Trang chủ của hệ thống gồm 2 thẻ là "Quản lý cuộc gặp của bản thân" và "Tham gia vào cuộc gặp khác". Thẻ mặc định được mở khi vào hệ thống là "Quản lý cuộc gặp của bản thân" gồm 2 phần là "Lên lịch một cuộc gặp" và "Danh sách cuộc gặp của bạn". Ngoài ra, người dùng có thể ẩn tạo

cuộc gặp ngay để tạo và tham gia ngay một cuộc gặp.



Hình 4.15: Giao diện lên lịch một cuộc gặp

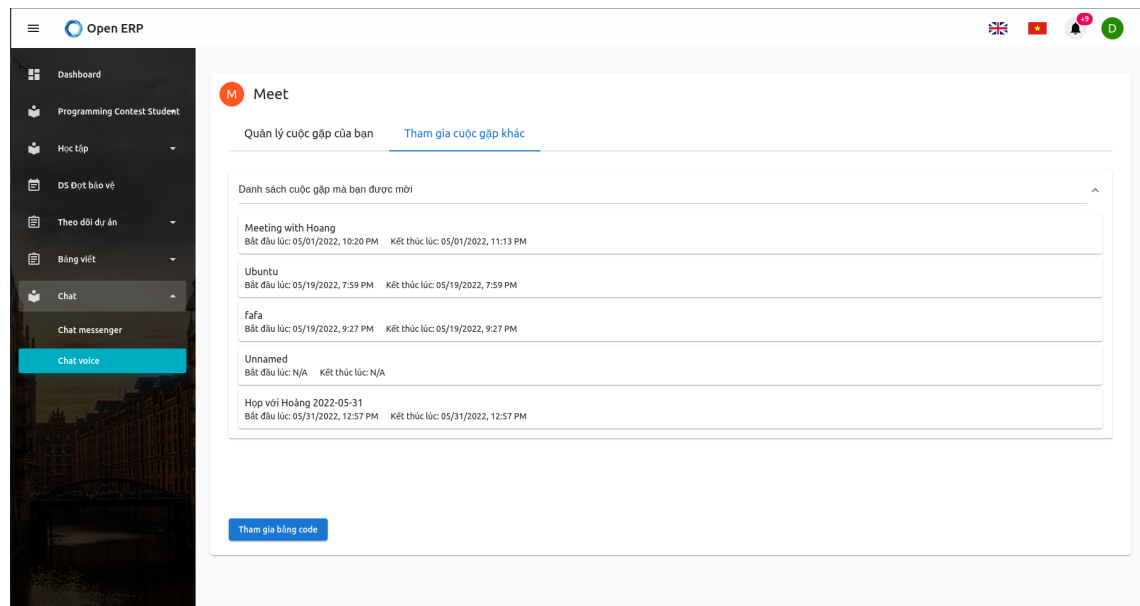
Giao diện lên lịch một cuộc gặp cho phép người dùng nhập tên cuộc gặp, thời gian bắt đầu, thời gian kết thúc. Sau khi nhập đầy đủ các trường, người dùng nhấn nút "Lên lịch" để tạo cuộc gặp.



Hình 4.16: Giao diện danh sách cuộc gặp của bạn

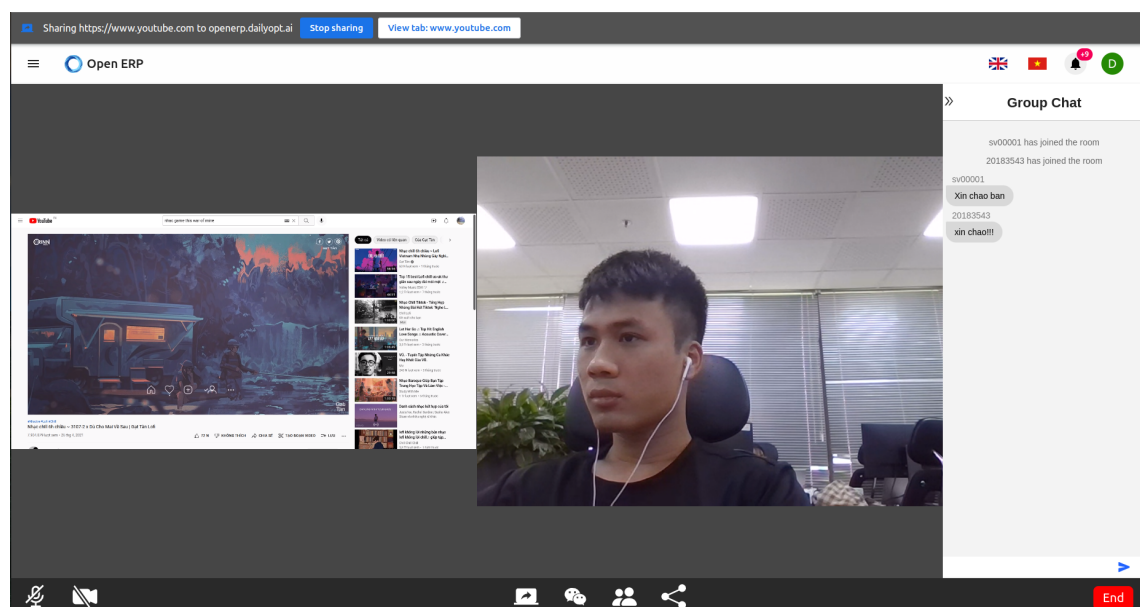
Giao diện danh sách cuộc gặp của bạn bao gồm một danh sách cuộc gặp mà người dùng tạo, được sắp xếp theo thứ tự thời gian tạo từ mới nhất đến cũ nhất. Mỗi một cuộc gặp bao gồm thông tin tên cuộc gặp, thời gian bắt đầu, thời gian kết thúc. Người dùng chọn một cuộc gặp trong danh sách nếu muốn tham gia hoặc sửa, xóa

cuộc gặp.



Hình 4.17: Giao diện Tham gia cuộc gặp khác

Giao diện Tham gia cuộc gặp khác gồm một danh sách các cuộc gặp mà người dùng được mời. Nhấn vào một cuộc gặp trong danh sách để tham gia vào cuộc gặp. Ngoài ra người dùng có thể nhấn vào nút "Tham gia bằng code", một hộp thoại sẽ hiển thị, người dùng nhập code của cuộc gặp là có thể tham gia vào cuộc gặp đó.



Hình 4.18: Giao diện màn hình cuộc gặp

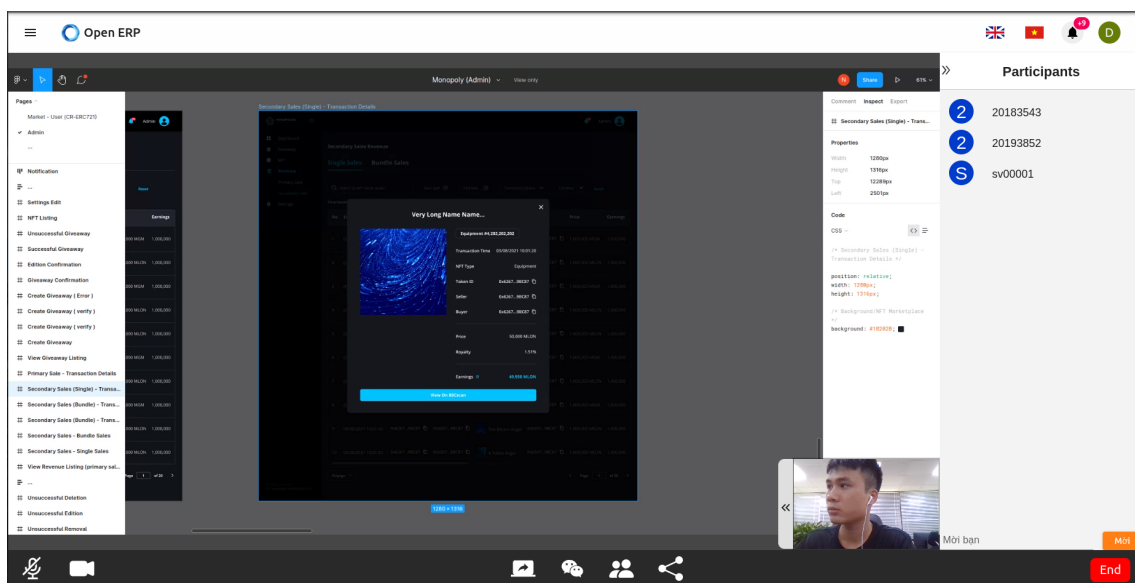
Hình 4.18 là giao diện của màn hình khi tham gia vào một cuộc gặp. Màn hình được chia làm 4 phần chính.

(i) Phía trên là header chung của toàn bộ dự án OpenERP.

(ii) Phía dưới là các nút cho phép người dùng thao tác với cuộc gọi, lần lượt từ trái qua phải là nút bật/tắt micro, bật/tắt camera, chia sẻ màn hình, mở thành group chat, mở thành quản lý danh sách người tham gia, chia sẻ cuộc gặp cho người khác và rời cuộc gặp.

(iii) Ở giữa là phần chính của cuộc gặp, đây là nơi hiển thị những hình ảnh từ camera, màn hình mà người tham gia chia sẻ với nhau, tại mỗi luồng hình ảnh sẽ hiển thị tên người tham gia và nút phóng to (khi nhấn vào sẽ mở rộng luồng hình ảnh của người này ra toàn bộ màn hình của bản thân) ở góc trái dưới.

(iv) Ở bên phải là khung chat của cuộc gặp, tại đây hệ thống sẽ hiển thị thông báo khi có người dùng khác tham gia hoặc rời khỏi cuộc gặp, hiển thị thông báo khi người dùng hiện tại tham gia và kết nối thành công vào cuộc gặp, tin nhắn/cuộc trò chuyện của người dùng hiện tại và những người tham gia cuộc gặp. Người dùng có thể nhắn tin/trò chuyện với mọi người bằng cách gõ tin nhắn và gửi ở phần phía dưới. Để ẩn thanh chat người dùng có thể ấn nút ».



Hình 4.19: Giao diện màn hình cuộc gặp với thẻ quản lý danh sách người tham gia

Hình 4.19 là giao diện màn hình khi tham gia một cuộc gặp với thẻ quản lý danh sách người tham gia đang mở ở bên phải. Tại đây người dùng có thể xem danh sách người dùng đang tham gia vào cuộc gặp và có thể sử dụng tính năng mời người dùng khác tham gia cuộc gặp tại góc phải dưới.

4.4 Kiểm thử

ID	Test case	Kết quả kỳ vọng	Kết quả thực tế
1	Để trống trường tên	Hiển thị thông báo chưa nhập tên	Đúng với kì vọng
2	Để trống trường bắt đầu lúc	Hiển thị thông báo chưa nhập thời điểm bắt đầu	Đúng với kì vọng
3	Để trống trường kết thúc lúc	Hiển thị thông báo chưa nhập thời điểm kết thúc	Đúng với kì vọng
4	Nhập tên có số kí tự lớn hơn 150	Hiển thị thông báo tên quá dài	Đúng với kì vọng
5	Nhập tên toàn kí tự khoảng trắng	Hiển thị thông báo chưa nhập tên	Đúng với kì vọng
6	Nhập tên gồm kí tự khác khoảng trắng, số kí tự nhỏ hơn bằng 150, các trường khác hợp lệ	Cuộc gặp được tạo thành công với trường bắt đầu lúc, kết thúc lúc, tên đã được loại bỏ kí tự khoảng trắng ở đầu và cuối như đã nhập	Đúng với kì vọng
7	Chọn bắt đầu lúc nhỏ hơn thời điểm hiện tại	Hiển thị thông báo thời gian lựa chọn không hợp lệ	Đúng với kì vọng
8	Chọn bắt đầu lúc lớn hơn thời điểm hiện tại và nhỏ hơn kết thúc lúc, tên hợp lệ	Cuộc gặp được tạo thành công với trường bắt đầu lúc, kết thúc lúc, tên đã được loại bỏ kí tự khoảng trắng ở đầu và cuối như đã nhập	Đúng với kì vọng
9	Chọn bắt đầu lúc lớn hơn thời điểm hiện tại và lớn hơn kết thúc lúc	Hiển thị thông báo thời gian lựa chọn không hợp lệ	Đúng với kì vọng

Bảng 4.8: Các trường hợp kiểm thử cho chức năng tạo cuộc gặp

Bảng 4.8 gồm 9 trường hợp kiểm thử cho chức năng tạo cuộc gặp. Các trường hợp kiểm thử này được xác định dựa trên kĩ thuật bảng quyết định (Decision table) của kiểm thử hộp trắng.

ID	Test case	Kết quả kỳ vọng	Kết quả thực tế
1	Để trống không nhập tên	Hiển thị thông báo chưa nhập tên	Đúng với kì vọng
2	Nhập tên người dùng không tồn tại	Hiển thị thông báo người dùng không tồn tại	Đúng với kì vọng
3	Nhập tên người dùng đã được mời tham gia cuộc gặp từ trước đó	Hiển thị thông báo người dùng đã được mời	Đúng với kì vọng
4	Nhập tên người dùng tồn tại trong hệ thống và chưa được mời tham gia cuộc gặp	Hiển thị thông báo mời thành công	Đúng với kì vọng

Bảng 4.9: Các trường hợp kiểm thử cho chức năng mời người dùng tham gia cuộc gặp

Bảng 4.9 gồm 4 trường hợp kiểm thử cho chức năng mời người dùng tham gia cuộc gặp. Các trường hợp này tương ứng với 4 phân vùng được xác định trong kĩ thuật phân vùng tương đương của kiểm thử hộp trắng.

STT	Chức năng	Số trường hợp kiểm thử	Kết quả thực tế (Thành công/Tổng số)
1	Tạo cuộc gặp	9	9/9
2	Mời người dùng tham gia cuộc gặp	4	4/4
Tổng		13	13/13

Bảng 4.10: Tổng hợp các trường hợp kiểm thử

4.5 Triển khai

Hệ thống được triển khai trên server có thông số và cấu hình như sau:

Thông số	Mô tả
CPU	Architecture x86_64
	CPU(s) 6
	Thread(s) per core 1
	Model name Intel(R) Xeon(R) CPU E5-2630 v4 @ 2.20GHz
RAM	8GB
HĐH	Distributor Ubuntu
	Description Ubuntu 18.04.5 LTS
	Release 18.04
	Codename bionic
Docker	Docker version 20.10.5
Image	Node 14.19.3-alpine
	Docker dind
	redis alpine
	postgreSQL latest
	mongo latest
	openjdk 8-jdk-alpine
Nginx	Version 1.21.6

Bảng 4.11: Thông số và cấu hình server

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Video Conference là một công nghệ còn nhiều mới lạ, tuy nhiên lại phát triển với rất nhanh. Dịch Covid-19 mang đến nhiều thách thức cho kinh tế và xã hội và Video Conference là một giải pháp giúp làm việc từ xa trở nên hiệu quả hơn. Chính vì thế chúng tôi đã quyết định phát triển hệ thống Video Conference, tức cuộc gặp trực tuyến của nhiều người. Rõ ràng có rất nhiều thách thức khi thực hiện đề tài này. Đầu tiên, có thể thấy Video Conference là một module vô cùng lớn với vô vàn tính năng. Ta có thể kể đến các chức năng như bật camera, micro để trò chuyện và tắt đi khi không cần thiết, chia sẻ màn hình để cùng theo dõi một công việc nào đó, chia sẻ tài liệu, nhắn tin, giơ tay, phân quyền người dùng (chủ phòng, người tham gia, người theo dõi,...), thực hiện bài tập trắc nghiệm ngay trên khung chat, giao bài tập về nhà, quay lại (record) cuộc gặp,... Tuy nhiên do thời gian và nguồn lực có hạn, chúng tôi phải chọn ra những chức năng cơ bản và cần thiết nhất để phát triển cho hệ thống Video Conference. Một trong những chức năng cơ bản và quan trọng nhất cũng như là chức năng khó nhất khi phát triển hệ thống này là chia sẻ camera, micro, màn hình realtime với tất cả người dùng trong cuộc gặp.

5.1 Chia sẻ camera, micro, màn hình thời gian thực với người khác

Chia sẻ camera, micro, màn hình realtime với người khác là chức năng quan trọng nhất của một hệ thống Video Conference. Để giải quyết bài toán trên ta cần giải quyết 3 bài toán con đó là lấy được dữ liệu hình ảnh từ camera, micro, màn hình của người dùng; chia sẻ với những người trong cùng cuộc gặp; đảm bảo tính thời gian thực cho luồng dữ liệu.

Với yêu cầu lấy luồng dữ liệu âm thanh, hình ảnh từ camera, micro và màn hình người dùng WebRTC là lựa chọn số một. WebRTC (Web Real-Time Communication) là một tập hợp các hàm lập trình (web API) được phát triển bởi World Wide Web Consortium (W3C). WebRTC hỗ trợ trình duyệt giao tiếp với nhau theo thời gian thực bằng video, âm thanh hay truyền tải dữ liệu "Peer-to-peer" (P2P) mà trình duyệt không cần cài đặt thêm bất kỳ một công cụ hỗ trợ bên ngoài nào khác. Chi tiết được nêu tại mục 5.2. Hiện nay WebRTC đã được tích hợp trong ngôn ngữ lập trình Javascript, trình duyệt có thể truy cập vào camera, micro, màn hình của người dùng để lấy ra dữ liệu hình ảnh và âm thanh. Ngoài WebRTC, FFmpeg [17], OBS [18] cũng là một công cụ giúp ta có thể trích xuất được dữ liệu hình ảnh, âm thanh từ thiết bị của người dùng. Tuy nhiên, đây đều là ứng dụng của bên thứ ba, người dùng cần cài đặt để có thể sử dụng được, việc tích hợp vào hệ thống đang có là phức tạp hơn rất nhiều. Điều này là hoàn toàn không cần thiết khi mà trình duyệt

và Javascript đã cũng cấp cho ta WebRTC vô cùng hoàn hảo và sẵn có để sử dụng.

Bài toán chia sẻ luồng dữ liệu này đến những người dùng khác cùng có mặt trong cuộc gặp là một bài toán khó khăn khác. Có hai hướng tiếp cận đối với bài toán này, một là kết nối tập trung, tức toàn bộ luồng dữ liệu hình ảnh, âm thanh của người dùng sẽ được xử lý tập trung tại server. Khi này server sẽ đóng vai trò trung chuyển dữ liệu giữa những người dùng, một người dùng sẽ tạo kết nối và chia sẻ luồng dữ liệu âm thanh, hình ảnh của bản thân cho server và nhận luồng dữ liệu của những người dùng khác từ server. Điểm mạnh khi phát triển hệ thống theo hướng này là mỗi người dùng sẽ chỉ cần tạo một kết nối duy nhất đến server, tại đây người dùng chỉ cần chia sẻ luồng dữ liệu của bản thân và nhận toàn bộ luồng dữ liệu của những người khác; toàn bộ các luồng dữ liệu sẽ tập trung tại server, server có thể toàn quyền xử lý các luồng dữ liệu người dùng, dễ dàng xử lý, áp dụng những tính năng như ghi hình cuộc gặp, cải thiện chất lượng hình ảnh, âm thanh cho các luồng dữ liệu người dùng,... Tuy nhiên, để nhận, chuyển và xử lý hàng loạt luồng dữ liệu âm thanh, hình ảnh theo thời gian thực là một chuyện không hề đơn giản. Khi số lượng người dùng ngày càng lớn lên, server càng cần xử lý nhiều luồng dữ liệu hơn. Rõ ràng, ta sẽ cần một hệ thống máy tính rất lớn để làm được công việc này. Một hướng tiếp cận khác với bài toán trên là kết nối phi tập trung P2P. Ngày nay, máy tính và các thiết bị điện tử cá nhân ngày càng phát triển, nhanh và mạnh mẽ hơn trước rất nhiều. Ta có thể tận dụng được điểm mạnh này để có thể phát triển hệ thống phi tập trung P2P. Điều này có nghĩa là mỗi người dùng sẽ tự kết nối với từng người dùng trong cùng một cuộc gặp, tự chia sẻ, nhận và xử lý những luồng dữ liệu của nhau. Ví dụ, A tham gia cuộc gặp gồm 6 người (bao gồm cả A), khi đó A sẽ tạo 5 kết nối với 5 người còn lại, mỗi kết nối sẽ xử lý truyền dữ liệu của A cho người còn lại và nhận dữ liệu của người còn lại cho A. Từ kết quả thực nghiệm trên chiếc máy tính cá nhân (Laptop Acer Nitro 5 AN515-54-51X1 NH.Q5ASV.011), người dùng có thể tạo kết nối P2P với 4 người mà chất lượng hình ảnh, âm thanh vẫn hoàn toàn mượt mà, không có cảm giác giật lác. Điểm mạnh của hướng tiếp cận này là khả năng mở rộng hệ thống theo chiều ngang, server sẽ không cần xử lý những luồng dữ liệu âm thanh, hình ảnh của người dùng, giảm tải rất lớn số lượng request, phần cần xử lý cho server nên số lượng cuộc gặp cùng diễn ra trên hệ thống sẽ lớn hơn, đáp ứng được số lượng người dùng cùng thời điểm nhiều hơn. Tuy nhiên, hướng tiếp cận này sẽ có hạn chế về khả năng mở rộng về chiều sâu, số lượng người dùng trong một cuộc gặp sẽ không thể quá nhiều do điều này là phụ thuộc vào thiết bị phần cứng của người dùng, khi số lượng người dùng trong một cuộc gặp nhiều hơn, một người dùng sẽ cần kết nối P2P đến nhiều người dùng hơn, máy tính của người dùng sẽ cần xử lý nhiều luồng dữ liệu hơn, gây ra hiện tượng

giật, lác, gây ra trải nghiệm không tốt. Do hạn chế về nguồn lực, việc sở hữu một server mạnh mẽ là bất khả thi, cùng với đó là mục tiêu và phạm vi phát triển là dành cho những nhóm nhỏ người dùng, chúng tôi đã lựa chọn hướng tiếp cận số hai để giải quyết bài toán trên.

Nhờ sử dụng WebRTC để truy xuất luồng dữ liệu âm thanh, hình ảnh thời gian thực từ micro, camera, màn hình của người dùng cùng với hướng tiếp cận kết nối phi tập trung P2P, luồng dữ liệu trên sẽ được truyền thẳng đến những người dùng khác với độ trễ rất nhỏ, bài toán về tính thời gian thực khi trò chuyện trực tuyến đã được giải quyết.

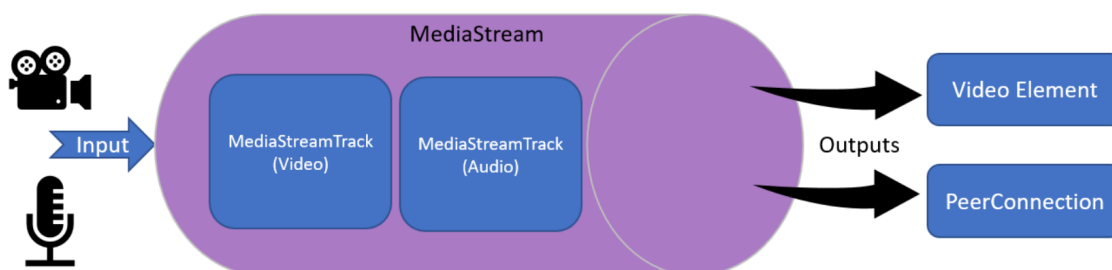
5.2 WebRTC

WebRTC (Web Real-time Communication) là một công nghệ Web mới cho phép trình duyệt và các ứng dụng di động có các chức năng gọi âm thanh/video, trò chuyện, chia sẻ tệp P2P mà không cần sử dụng bất kỳ phần mềm hoặc tiện ích bên thứ ba nào [19].

WebRTC được Google công bố dưới dạng công nghệ mã nguồn mở vào tháng 5 năm 2011 và bao gồm các thành phần cơ bản để giao tiếp thời gian thực trên web. Google Chrome, Mozilla, Opera đã hỗ trợ webRTC và chúng sẽ góp phần vào việc phát triển công nghệ này hơn nữa. Các thành phần cơ bản để giao tiếp thời gian thực trên web được truy cập thông qua Javascript API, có thể kể đến như:

- (i) `MediaStream`: Cho phép trình duyệt web truy cập máy ảnh và micro [20].
- (ii) `RTCPeerConnection`: Thiết lập cuộc gọi âm thanh hoặc video [21].
- (iii) `RTCDataChannel`: Cho phép các trình duyệt gửi dữ liệu thông qua kết nối P2P [22].

5.2.1 MediaStream



Hình 5.1: Media Stream

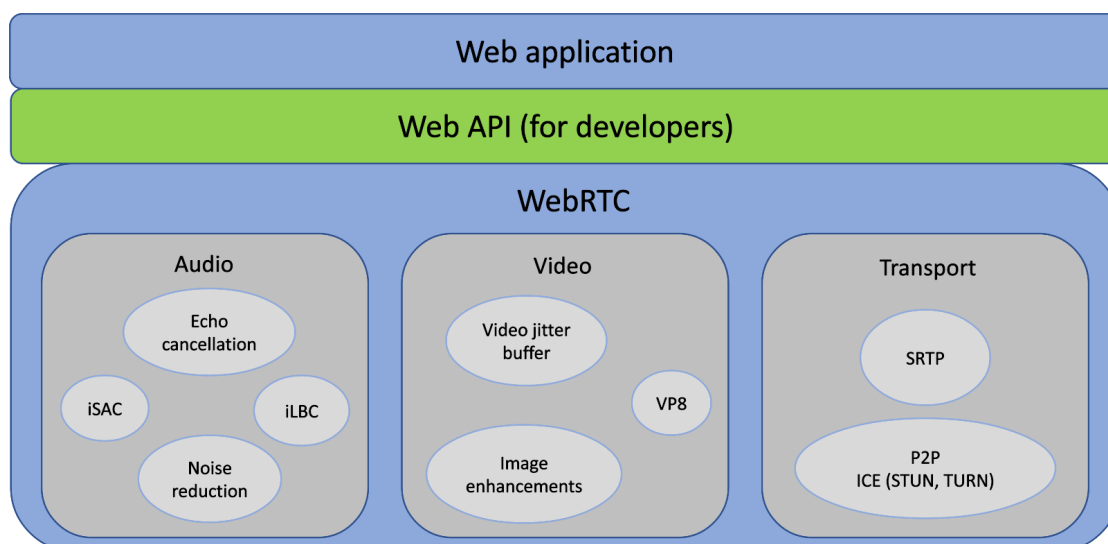
`MediaStream` API có quyền kiểm soát các luồng phương tiện được đồng bộ hóa [23]. Mỗi một `MediaStream` có một đầu vào và đầu ra. Đầu vào có thể là các thiết bị vật lý như máy ảnh, micro. Đầu ra có thể chuyển tiếp đến một hoặc nhiều đích

đến, chẳng hạn như thẻ video trong HTML hoặc PeerConnection (Hình 5.1).

Hai thành phần chính trong MediaStream API là MediaStream Interface và MediaStreamTrack. MediaStream Interface đại diện cho một luồng nội dung đa phương tiện. Luồng đa phương tiện này có thể có nhiều phần. Những phần đó được gọi là tracks và chúng có loại rõ ràng, ví dụ như audio hoặc video. MediaStreamTrack đại diện cho một loại đa phương tiện được lấy từ nguồn đầu vào. Ví dụ, nếu đó là micro, chúng ra sẽ nhận được MediaStreamTrack (Audio) [19].

5.2.2 RTCPeerConnection

RTCPeerConnection API cho phép 2 peer giao tiếp trực tiếp, giữa trình duyệt với trình duyệt. Sơ đồ kiến trúc WebRTC (Hình 5.2) thể hiện một hệ thống phân cấp rất đơn giản. WebRTC cung cấp một khuôn khổ để thực hiện giao tiếp theo thời gian thực.



Hình 5.2: WebRTC architecture

Kiến trúc WebRTC có ba phần cơ bản. Phần đầu tiên, WebRTC, bao gồm một cơ chế audio, video và transport.

Nhiệm vụ chính của cơ chế audio là xử lý "echo cancellation" và "noise reduction". Echo cancellation là một phương thức loại bỏ tiếng vọng hoặc tránh để tiếng vọng xuất hiện. Noise reduction là một quy trình để giảm tiếng ồn của tín hiệu âm thanh. Cơ chế audio bao gồm iSAC và iLBC codecs (codec là viết tắt của compressor/decompressor hay coder/decoder, là một công cụ mã hóa, xử lý video/audio và lưu trữ nó trong một luồng byte) đều được phát triển bởi Global IP Solutions. iSAC codec là codec âm thanh băng tần rộng phù hợp với luồng âm thanh, trong khi iLBC codec là codec cho âm thanh băng tần hẹp phù hợp với giao tiếp thoại qua IP.

Cơ chế video có nhiệm vụ là "image enhancement" và "video jitter buffer". Image enhancement là quá trình cải thiện chất lượng của hình ảnh kỹ thuật số bằng cách thao tác với một phần mềm nhất định. Cơ chế video bao gồm VP8 codec. VP8 codec là một định dạng nén video được phát triển bởi On2 Technologies vào tháng 9 năm 2008 và hiện vẫn đang hỗ trợ bởi Chrome, Mozilla và Opera.

Cơ chế transport chịu trách nhiệm đảm bảo tính an toàn khi truyền tải thời gian thực bằng SRTC (Secure Real-time Transport Protocol) và thiết lập kết nối tương tác P2P và ICE. P2P và ICE với STUN và TURN server sẽ được giải thích ở phần 5.3.

Phần tiếp sau của kiến trúc là phần Web API. Tại phần này, hệ thống Video Conference sử dụng thư viện mã nguồn mở của Javascript là PeerJs để đơn giản hóa công việc. PeerJs cung cấp một API P2P hoàn chỉnh, có thể định cấu hình, dễ sử dụng được xây dựng trên nền WebRTC, hỗ trợ cả kênh dữ liệu và luồng phương tiện. Phần thứ ba, phần cuối cùng là Web Application. Đây chính là những ứng dụng âm thanh, video, trò chuyện trực tuyến.

5.2.3 RTCDataChannel

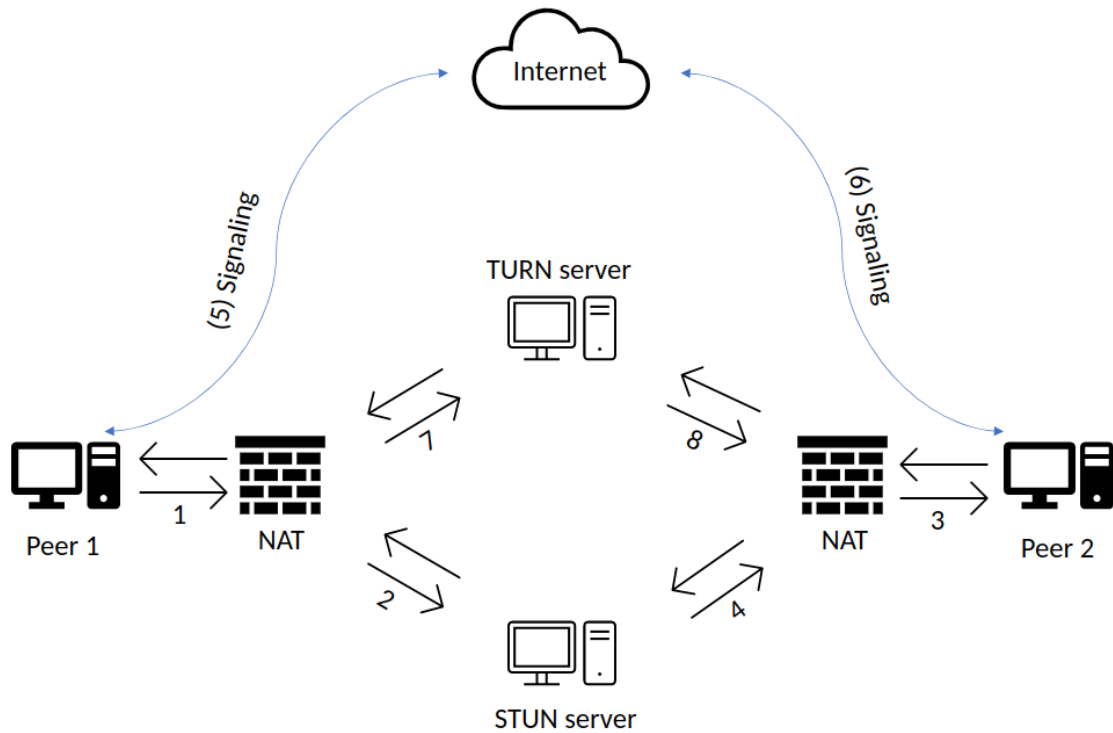
RTCDataChannel API đại diện cho một kênh dữ liệu hai chiều giữa 2 peer và cho phép trao đổi dữ liệu tùy ý giữa chúng. Mỗi RTCDataChannel có thể cấu hình để cung cấp:

- (i) cung cấp thông điệp đáng tin cậy hoặc không đáng tin cậy.
- (ii) Phân phối tin nhắn theo thứ tự hoặc không theo thứ tự.

Việc phân phối tin nhắn theo thứ tự và đáng tin cậy tương đương với TCP (Transmission Control Protocol). Mặt khác, việc phân phối không theo thứ tự và không đáng tin cậy tương đương với UDP (User Datagram Protocol). Có rất nhiều tiềm năng sử dụng với API này, bao gồm:

- gaming;
- remote desktop applications;
- real-time text chat;
- file transfer.

5.3 P2P và ICE



Hình 5.3: Interactive Connectivity Establishment (ICE)

Hệ thống định địa chỉ Internet vẫn đang sử dụng IPv4 (Internet Protocol version 4) và do đó, hầu hết các thiết bị của chúng ta đều có một hoặc nhiều lớp NAT (Network Address Translation). NAT là một cơ chế ánh xạ địa chỉ private đến địa chỉ công cộng, thay đổi thông tin địa chỉ bên trong các gói IP khi nó được truyền qua thiết bị để định tuyến. NAT có thể chính là bộ định tuyến Wifi trong gia đình.

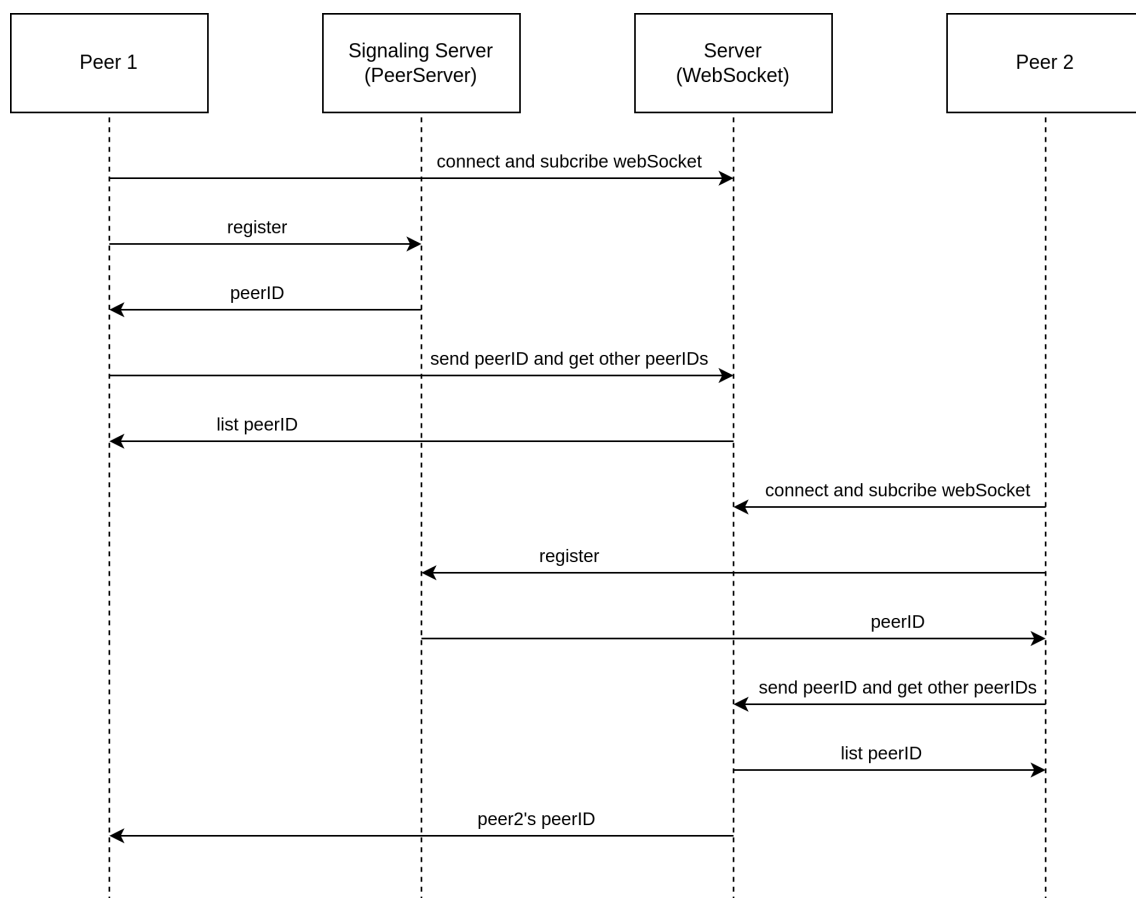
Các nhà phát triển công nghệ WebRTC có thể sử dụng ICE để đơn giản hóa độ phức tạp của hệ thống định địa chỉ Internet. Các nhà phát triển cần chuyển các URL máy chủ ICE tới kết nối peer. ICE sẽ cố gắng tìm ra con đường tốt nhất để kết nối 2 peer (Hình 5.3). ICE đang sử dụng hai loại server, STUN (Session Traversal Utilities for NAT) và TURN (Traversal Using Relays around NAT). ICE sử dụng STUN server để tìm ra địa chỉ bên ngoài nào được gán cho một peer cụ thể. Nếu không thành công, ICE sẽ sử dụng TURN server để định tuyến. Mọi TURN server đều hỗ trợ STUN, TURN server là một STUN server được tích hợp thêm chức năng chuyển tiếp [24].

Hiện nay, có rất nhiều STUN server, TURN server cho phép lập trình viên sử dụng hoàn toàn miễn phí. Ban đầu hệ thống Video Conference sử dụng server TURN do Xirsys [25] cung cấp. Tuy nhiên trong quá trình sử dụng, các cuộc gặp trong hệ thống đôi khi không kết nối được với nhau, sau khi tìm hiểu, nguyên nhân

xuất phát từ TURN server này. Chính vì vậy, chúng tôi tự xây dựng một TURN server được phát triển bằng thư viện Javascript, PeerServer. PeerServer là một thư viện Javascript chạy trên NodeJs, giúp thiết lập kết nối giữa các PeerJs client.

5.4 Mô hình Video Call thông qua WebRTC

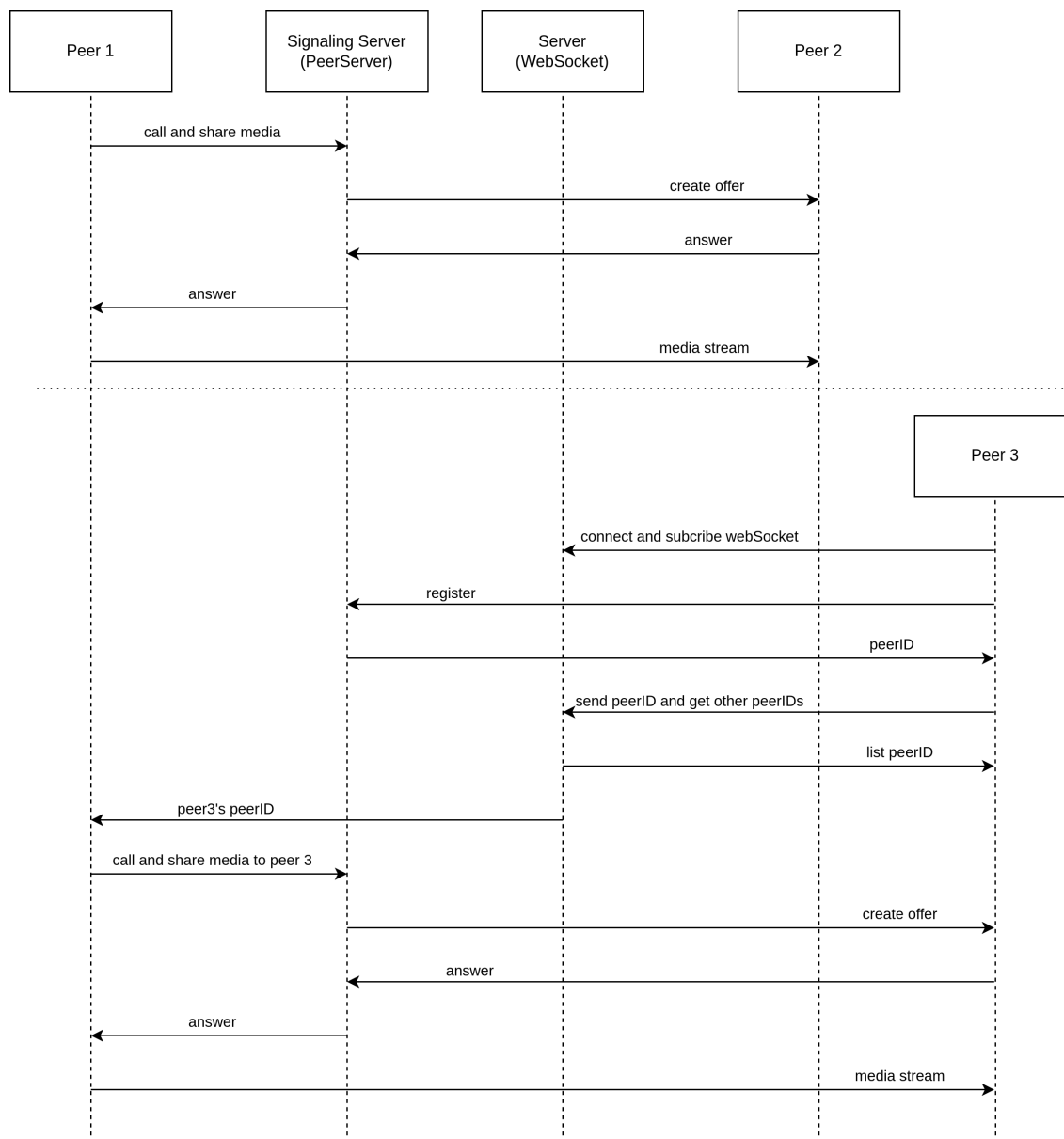
Trên cơ sở lý thuyết được nêu tại phần 5.2, tại phần này, chúng tôi trình bày về mô hình video call được áp dụng với những cuộc gặp trong hệ thống. Mô hình gồm 3 thành phần chính là server được tích hợp websocket, signaling server và peer (người dùng trong cuộc gặp). Do sử dụng kết nối ngang hàng P2P nên vai trò của người dùng trong một cuộc gặp là tương đương nhau.



Hình 5.4: Đăng kí và chia sẻ mã định danh peerID

Hình 5.4 là biểu đồ thể hiện quy trình đăng kí mã định danh peerID với signaling server và chia sẻ mã này với những người dùng trong cùng một cuộc gặp. Khi tham gia vào một cuộc gặp, người dùng sẽ kết nối và đăng kí lắng nghe các sự kiện của websocket. Sau đó, người dùng sẽ đăng kí mã định danh với signaling server. Signaling server sẽ tạo ra một mã duy nhất peerID, là định danh của người dùng, và gửi lại cho người dùng. Người dùng gửi mã định danh này tới server, đồng thời yêu cầu lấy danh sách mã định danh peerID của những người đang tham gia vào

cuộc gặp. Server sẽ lưu lại peerID của người dùng và phản hồi lại yêu cầu của người dùng một danh sách các peerID của những người dùng trong cuộc gặp, đồng thời server sẽ gửi mã định danh của người này cho tất cả những người đang tham gia cuộc gặp này. Quy trình này được áp dụng với tất cả những người tham gia vào cuộc gặp.



Hình 5.5: Kết nối P2P và chia sẻ luồng dữ liệu đa phương tiện với những người tham gia khác trong cuộc gặp

Hình 5.5 là biểu đồ thể hiện quy trình kết nối P2P và chia sẻ luồng dữ liệu đa phương tiện cho những người tham gia khác trong cuộc gặp mặt sau khi đã đăng kí với signaling server peerID của bản thân và nhận được danh sách peerID của những người tham gia khác trong cuộc gặp từ server. Khi người dùng (gọi là peer 1) bật camera hoặc bật mic hoặc chia sẻ màn hình, nhờ webRTC một luồng dữ liệu đa

phương tiện sẽ được tạo ra, đồng thời gửi yêu cầu kết nối P2P và chia sẻ luồng dữ liệu này với lần lượt từng peerID ứng với mỗi người dùng đang có mặt trong cuộc gặp (gọi là peer 2) cho signaling server. Nhờ peerID đã được đăng kí, signaling server sẽ nhận ra và tìm đường kết nối tốt nhất giữa 2 peer. Signaling server sẽ thông báo với peer 2 yêu cầu kết nối từ peer 1. Peer 2 sẽ phản hồi đồng ý kết nối với signaling server, signaling chuyển tiếp phản hồi này lại cho peer 1. Cuối cùng 2 peer sẽ tạo kết nối với nhau bằng RTCPeerConnection, peer 1 sẽ gửi luồng dữ liệu đa phương tiện của bản thân cho peer 2. Qua một vài xử lý đơn giản, peer 2 có thể xem và nghe được âm thanh, hình ảnh từ luồng dữ liệu trên. Trong hệ thống Video Conference, theo nghiệp vụ mỗi khi nhận được thông báo kết nối, các peer sẽ luôn đồng ý kết nối, tuy nhiên trong các ứng dụng khác, các peer hoàn toàn có thể từ chối kết nối.

Đối với một người dùng tham gia vào cuộc gặp muộn hơn, khi peer 1 đang chia sẻ mediaStream của bản thân (gọi là peer 3), thì người này cũng cần nhận được luồng dữ liệu của peer 1. Điều này giống với việc bạn đang học thì đi ra ngoài sau đó vào lại lớp học hoặc đơn giản là bạn đến lớp muộn hơn giáo viên thì vẫn phải nghe, xem được những gì mà giáo viên đang giảng vậy. Hệ thống Video Conference cũng hoạt động như vậy. Đối với người dùng tham gia vào cuộc gặp muộn hơn, sau khi đăng kí peerID với signaling server và gửi mã định danh này cho server như tất cả những người tham gia khác, server sẽ gửi peerID của người này cho peer 1. Với mediaStream hiện có và peerID của peer 3, peer 1 sẽ lại gửi yêu cầu kết nối P2P và chia sẻ luồng dữ liệu này với signaling server, signaling server tiếp tục tìm đường kết nối tốt nhất giữa 2 peer và gửi thông báo tới peer 3. Peer 3 đồng ý và kết nối giữa 2 peer được thiết lập. Peer 3 có thể xem, nghe được những gì peer 1 đang chia sẻ. Như vậy, hệ thống Video Conference đã đảm bảo, dù trong trường hợp đến sớm hay đến muộn, rồi cuộc họp sau đó vào lại thì kết nối giữa mọi người dùng trong cùng cuộc gặp luôn được thiết lập.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Nhu cầu họp hành cũng như đào tạo từ xa ngày càng trở nên cấp thiết. Đặc biệt, các tổ chức doanh nghiệp lớn luôn có mong muốn làm chủ công nghệ lõi và xây dựng các hệ thống riêng đáp ứng các yêu cầu đặc thù. Với bối cảnh đó, mục tiêu của đồ án tốt nghiệp là thiết kế và xây dựng một hệ thống Video Conference cho phép người dùng có thể kết nối trò chuyện thông qua hình ảnh, âm thanh để có thể áp dụng vào các nghiệp vụ trong tổ chức doanh nghiệp như hội thảo trực tuyến, đào tạo từ xa.

Kết quả đạt được của đồ án là một hệ thống Video Conference được xây dựng dựa trên công nghệ WebRTC, SpringBoot, ReactJS với các tính năng cho phép người dùng kết nối trò chuyện qua âm thanh hình ảnh, chia sẻ màn hình, nhắn tin, chia sẻ file trong thời gian thực. Hệ thống đã được triển khai chạy thử nghiệm và sử dụng nội bộ trong nhóm phát triển dự án ERP [6].

6.2 Hướng phát triển

Cho đến thời điểm hiện tại, những chức năng cho hệ thống Video Conference đề ra ban đầu đều đã được hoàn thành. Tuy nhiên, không phải chức năng nào cũng được kiểm thử đầy đủ. Một số lỗi đã được phát hiện nhưng chưa được sửa như khi một người tắt camera thì phần video của người đó không được tắt đi mà vẫn còn lưu lại hình ảnh cũ. Trong thời gian tới, nhiệm vụ của chúng tôi cần làm là kiểm thử và sửa những lỗi của hệ thống. Tiếp đến, việc mỗi cuộc gặp chỉ có thể có tối đa 5 người là một nhược điểm cần cải thiện thêm. Trong tương lai chúng tôi mong muốn số người trong cuộc gặp tăng lên tối đa 20 người. Để làm được điều này, chúng tôi cần sử dụng ReactJs để chia màn hình người dùng ra thành nhiều phần để chứa những luồng video chuyển đến từ người dùng khác sao cho hợp lý (Khi 10 người thì hiển thị như thế nào? Khi 11 người thì hiển thị như thế nào?). Xa hơn là giải quyết bài toán, cho X người, chia màn hình thành X phần để hiển thị. Một số chức năng như record lại cuộc gặp; một người có thể đồng thời chia sẻ màn hình, chia sẻ camera; Điều khiển chuột, bàn phím từ xa (giống TeamViewer); công nghệ làm đẹp khuôn mặt người dùng nhờ công nghệ nhận diện khuôn mặt; thêm nhãn dán cho cuộc trò chuyện thêm sinh động; lưu lại lịch sử trò chuyện, lịch sử chia sẻ file; ... là những chức năng hay, cần có cho hệ thống, nếu có thể phát triển thành công sẽ đem lại trải nghiệm tốt hơn với người dùng.

TÀI LIỆU THAM KHẢO

- [1] *Microsoft teams*. [Online]. Available: <https://teams.microsoft.com/>.
- [2] *Zoom*. [Online]. Available: <https://zoom.us/>.
- [3] *Google meet*. [Online]. Available: <https://meet.google.com/>.
- [4] *Zalo*. [Online]. Available: <https://zalo.me/pc>.
- [5] *Facebook*. [Online]. Available: <https://facebook.com>.
- [6] *Open erp*. [Online]. Available: <https://github.com/dungkhmt/openerp>.
- [7] R. T. Fielding, *Architectural styles and the design of network-based software architectures*. University of California, Irvine, 2000.
- [8] *Requirements for using google meet*. [Online]. Available: <https://support.google.com/meet/answer/7317473?hl=en#zippy=%2Cparticipant-limits>.
- [9] *Advantages java*. [Online]. Available: <https://www.ibm.com/docs/en/aix/7.1?topic=monitoring-advantages-java>.
- [10] *String boot*. [Online]. Available: <https://spring.io/projects/spring-boot>.
- [11] *Reactjs*. [Online]. Available: <https://reactjs.org/docs/getting-started.html>.
- [12] *Introducing hooks*. [Online]. Available: <https://reactjs.org/docs/hooks-intro.html>.
- [13] *Postgresql*. [Online]. Available: <https://www.postgresql.org/about/>.
- [14] V. Pimentel and B. G. Nickerson, “Communicating and displaying real-time data with websocket,” *IEEE Internet Computing*, vol. 16, no. 4, pp. 45–53, 2012.
- [15] *Node.js*. [Online]. Available: <https://nodejs.dev/learn>.
- [16] A. Aarsten, D. Brugali, and G. Menga, “Patterns for three-tier client/server applications,” *Proceedings of Pattern Languages of Programs (PLoP’96)*, vol. 3, no. 6, 1996.
- [17] *Ffmpeg*. [Online]. Available: <http://ffmpeg.org/>.
- [18] *Obs*. [Online]. Available: <https://obsproject.com/>.
- [19] B. Sredojev, D. Samardzija, and D. Posarac, “Webrtc technology overview and signaling solution design and implementation,” in *2015 38th international convention on information and communication technology, electronics*

- and microelectronics (MIPRO)*, IEEE, 2015, pp. 1006–1009.
- [20] *Media stream*. [Online]. Available: <https://webrtc.org/getting-started/media-devices>.
- [21] *Rtc peer connection*. [Online]. Available: <https://webrtc.org/getting-started/peer-connections>.
- [22] *Rtc data channel*. [Online]. Available: <https://webrtc.org/getting-started/data-channels>.
- [23] *Permissions integration*. [Online]. Available: <https://w3c.github.io/mediacapture-main/getusermedia>.
- [24] A. Johnston, J. Yoakum, and K. Singh, “Taking on webrtc in an enterprise,” *IEEE Communications Magazine*, vol. 51, no. 4, pp. 48–54, 2013.
- [25] *Turn server cloud*. [Online]. Available: <https://xirsys.com/>.